Presentation 1

STAT 390 Fall 2026

Team 2 · The Consultancy

Lauren Vegas

Callie Berthold

Lily Wu

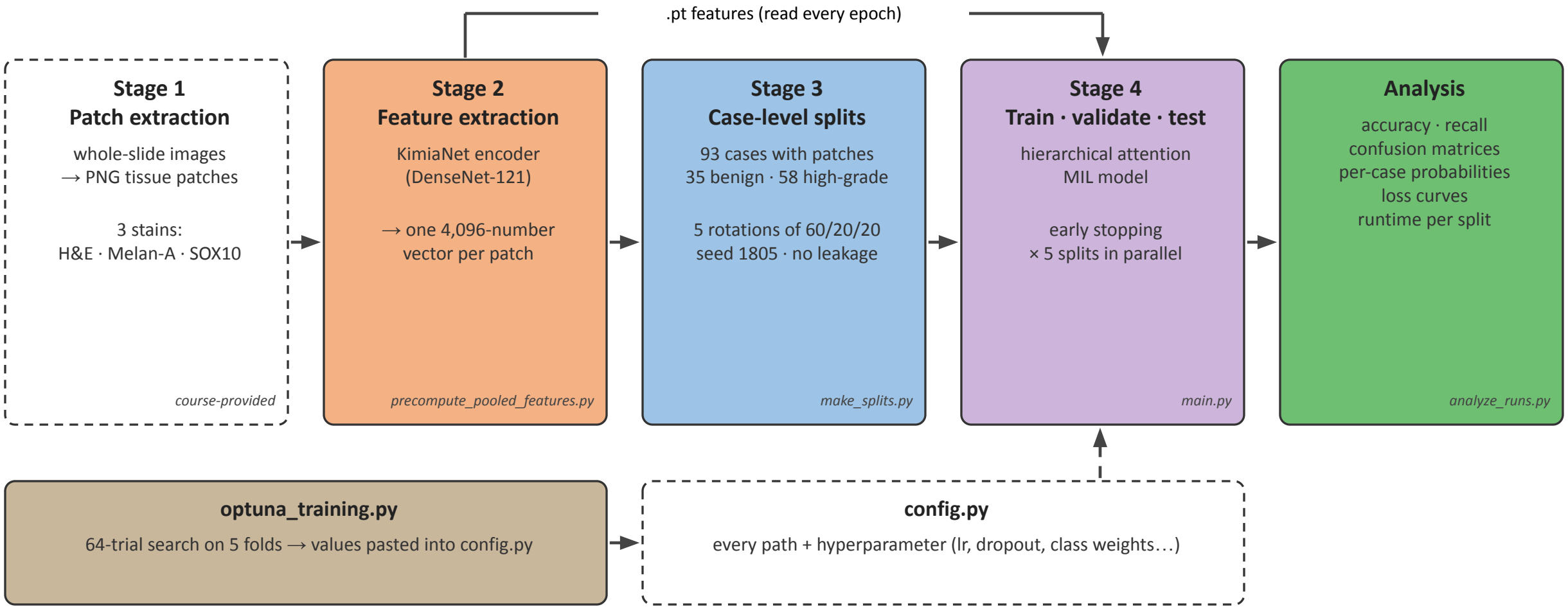
Alyssa Kantner

Izabella Kulczycka

The Pipeline

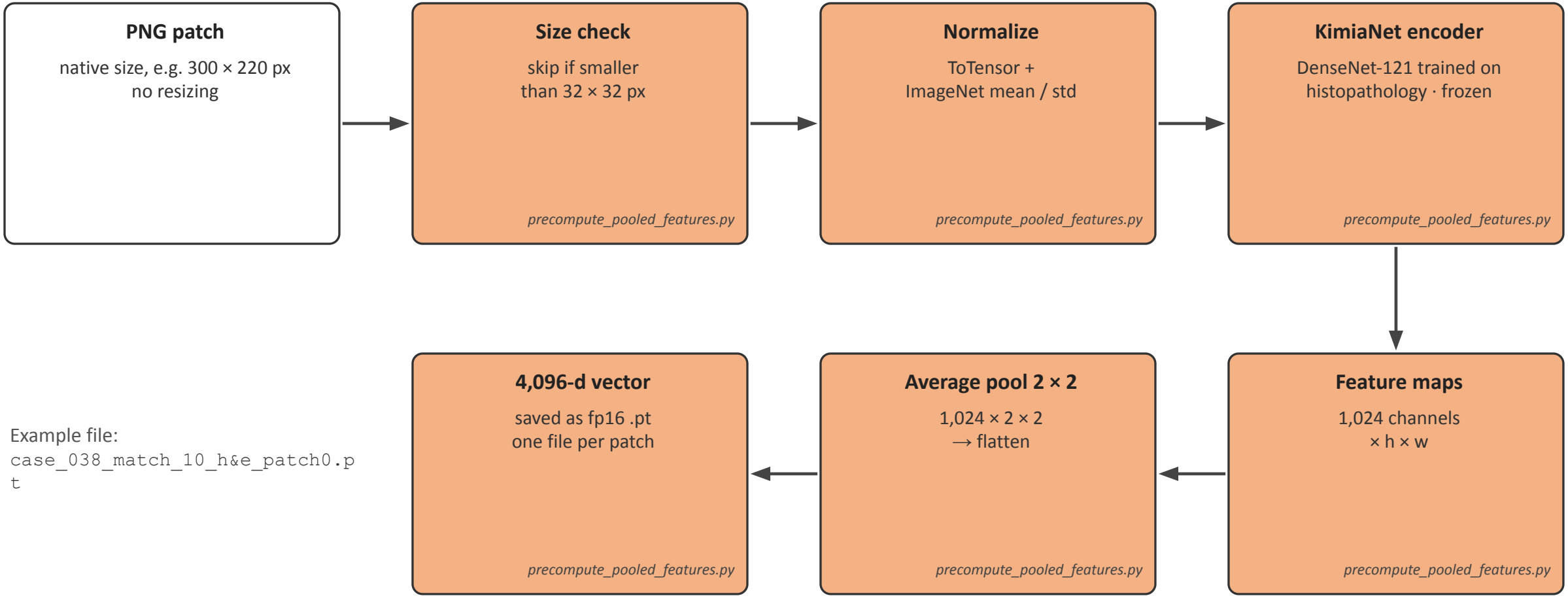
The pipeline at a glance

Stages 1–2 were run once by the course; this project runs Stages 3–4 on 5 splits and analyzes them



Stage 2 · Turning every patch into 4,096 numbers

Run once for all patches; every split and every epoch reuses these files



Example file:
`case_038_match_10_h&e_patch0.pt`

Patches have different sizes, so they go through the encoder one at a time (no batching).

Stage 3 · Five leak-free splits from one seed

Split by case (patient), never by patch, so no test case is ever seen in training

load_labels

keep Class 1, 3, 4
1 → benign · 3, 4 → high-grade

data_utils.py

group_patches_by_slice

parse case_<id>_<slice>_<stain>…
→ (case, slice) → patch list

data_utils.py

93 cases

35 benign · 58 high-grade
related benign cases share a group

make_splits.py

Ratios → folds

60 / 20 / 20 = 3 : 1 : 1
→ 5 folds

make_splits.py

Seed 1805

randomly relabels group IDs
(shuffle stays off)

make_splits.py

StratifiedGroupKFold(5)

same class mix in every fold
related cases stay together

make_splits.py

Rotate folds: each is test once

	Fold A	Fold B	Fold C	Fold D	Fold E
Split 01	train	train	train	val	**test**
Split 02	**test**	train	train	train	val
Split 03	val	**test**	train	train	train
Split 04	train	val	**test**	train	train
Split 05	train	train	val	**test**	train

report_no_leak → 5 × .npz

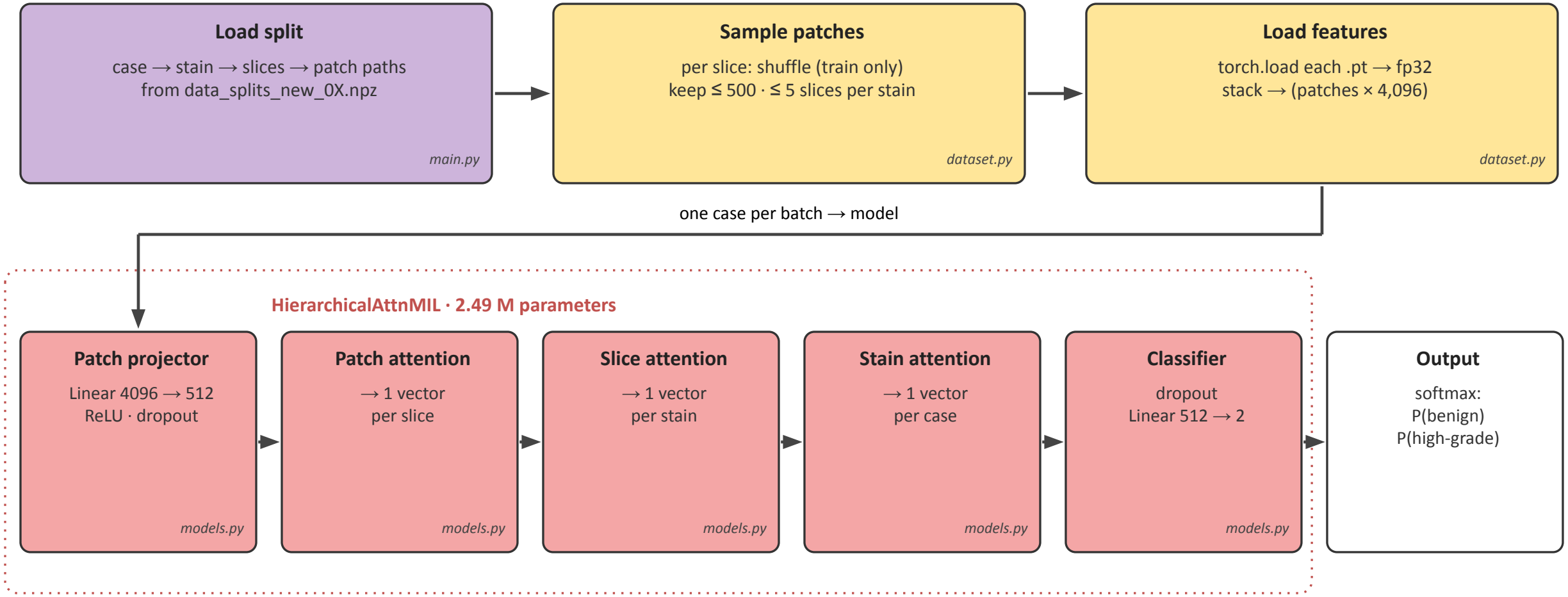
case- and patch-level overlap checks
data_splits_new_01 … 05.npz

data_utils.py · utils.py

Box color = script: data_utils.py | make_splits.py | utils.py

Stage 4a · One case in, one prediction out

Three levels of attention: patches → slice, slices → stain, stains → case

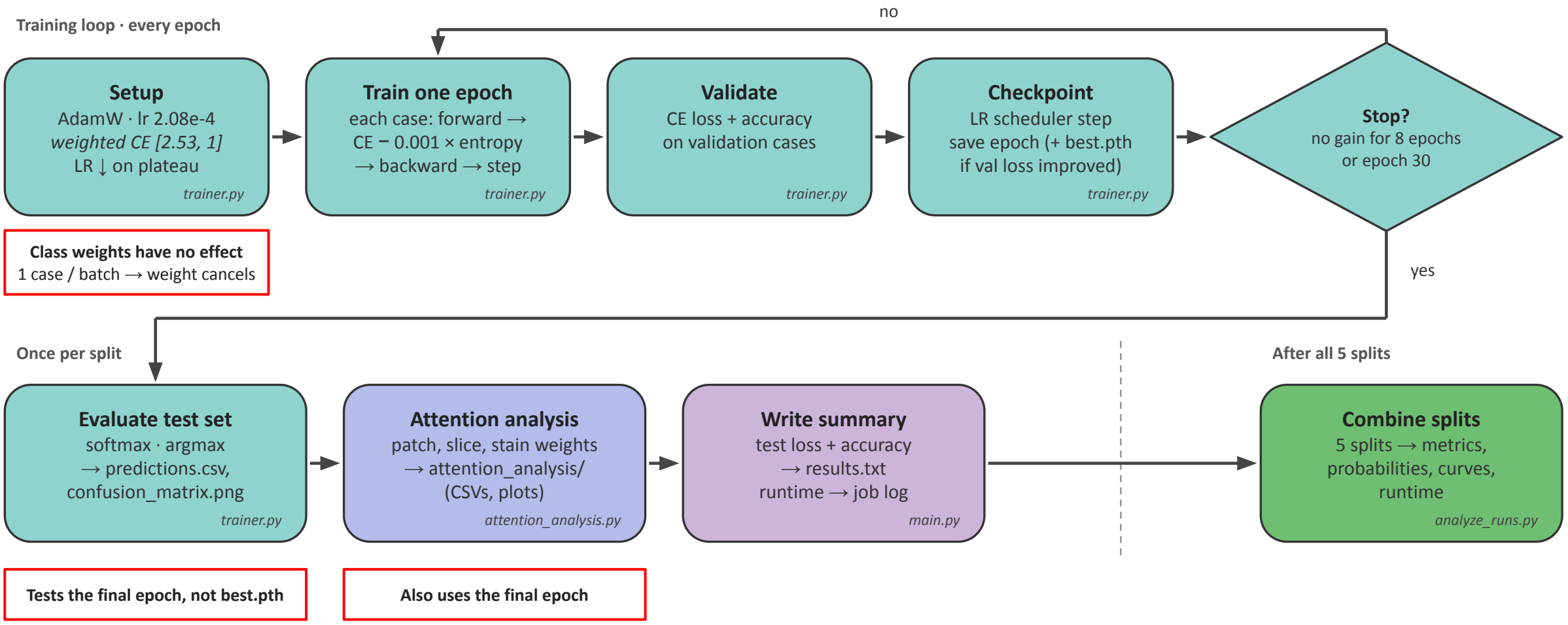


gated attention: $score = w^T[\ \tanh(V \cdot h) \odot \mathrm{sigmoid}(U \cdot h)\ ] \rightarrow weights = \mathrm{softmax}(scores) \rightarrow \Sigma\ weight \cdot h$

Box color = script: main.py | dataset.py | models.py

Stage 4b · Training loop, early stopping and test evaluation

Repeated for each of the 5 splits; the analysis script combines them



Box color = script: trainer.py | attention_analysis.py | main.py | analyze_runs.py

Results and Reflection

Execute · 5 splits on Quest with my own seed (1805)

Stages 3 and 4 run end to end; every result is reproducible from one number

Problem found: the seed was ignored

make_splits.py accepts --seed but never uses it.
With shuffle off, sklearn orders folds by group ID,
so seeds 1 and 999 gave the identical test set:
every student had the same 5 splits.

Fix: seed relabels group IDs (shuffle stays off)

```
uniq = np.unique(groups)
new_id = dict(zip(uniq,
    np.random.RandomState(seed).permutation(len(uniq))))
groups = np.array([new_id[g] for g in groups])
```

1 · split_script.sbatch

make_splits.py --seed 1805
→ 5 leak-free .npz splits
93 cases · 35 benign · 58 high

2 · train_5_splits.sbatch

SLURM array 1–5, in parallel
main.py --load_splits <split>
--seed 1805 · timed per split

3 · analyze.sbatch

analyze_runs.py → metrics,
probabilities, loss curves,
runtime for all 5 splits

Checks: no case or patch overlap between train / val / test · each case tested exactly once · high-grade share 0.61–0.63 in every test set

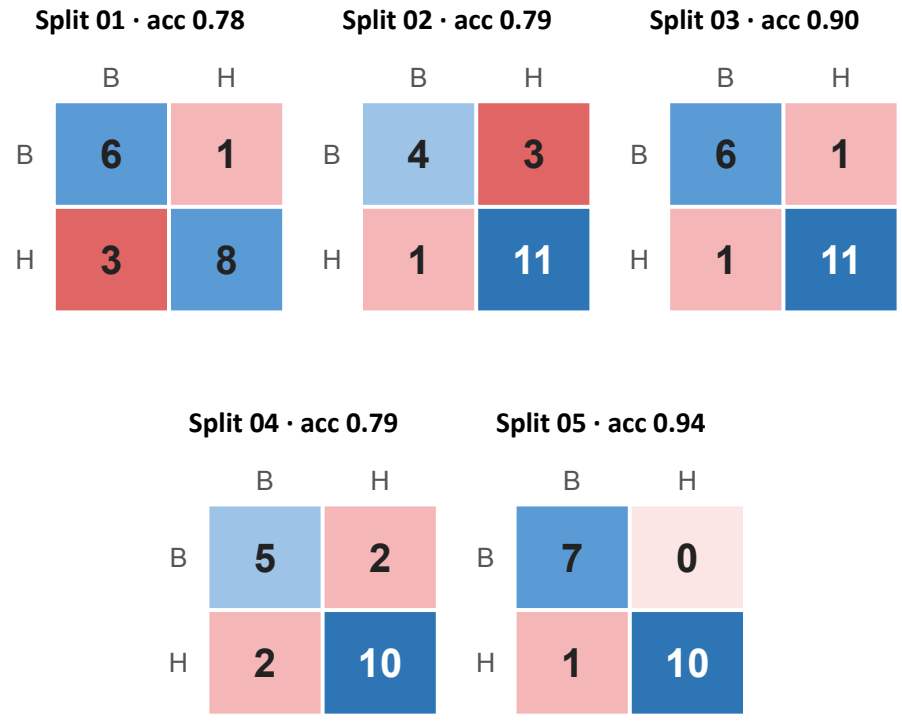
Results · accuracy, recall and confusion matrices

Seed 1805 · 5 splits · 93 test cases in total (each case tested once)

Split	Test cases	Accuracy	Recall benign	Recall high-grade
01	18	0.778	0.857	0.727
02	19	0.789	0.571	0.917
03	19	0.895	0.857	0.917
04	19	0.789	0.714	0.833
05	18	0.944	1.000	0.909
Mean ± SD	**18.6**	**0.839 ± 0.076**	**0.800 ± 0.163**	**0.861 ± 0.082**
Pooled	**93**	**78/93 = 0.839**	**28/35 = 0.800**	**50/58 = 0.862**

Benign recall swings most (0.571–1.000): only 7 benign cases per test set, so one error moves it by 14 points. Split 02 had 3 of the 7 benign errors.
Split 01 had 3 of the 8 missed high-grade cases. With 18–19 test cases, one case changes accuracy by about 5 points.

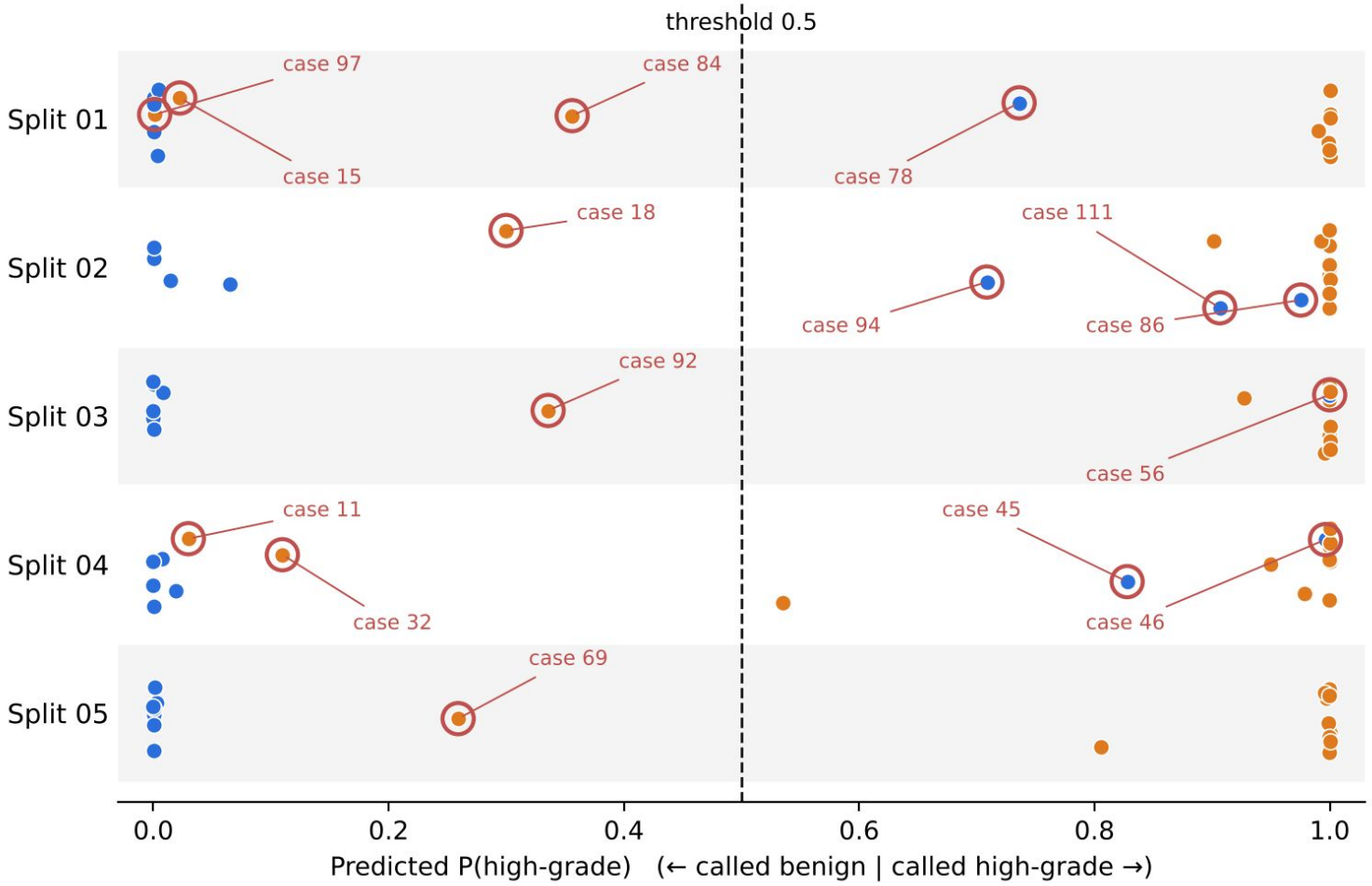
Confusion matrices (rows = true, columns = predicted)



B = benign, H = high-grade · blue = correct · red = error
Top-right red = false alarm (benign → high) · bottom-left red = miss (high → benign)

Results · predicted probability for each test case

Each dot is one test case (93 total); the model calls high-grade when P(high-grade) > 0.5



truly benign

truly high-grade

misclassified (15 of 93)

Very confident when right

76 of 78 correct cases had ≥ 0.9 probability on the true class.

Often confident when wrong

7 of 15 errors were beyond 0.1 / 0.9 (e.g. benign case 56 at 0.99997).

Ranking is strong

Pooled AUC = 0.94: high-grade cases usually score above benign ones.

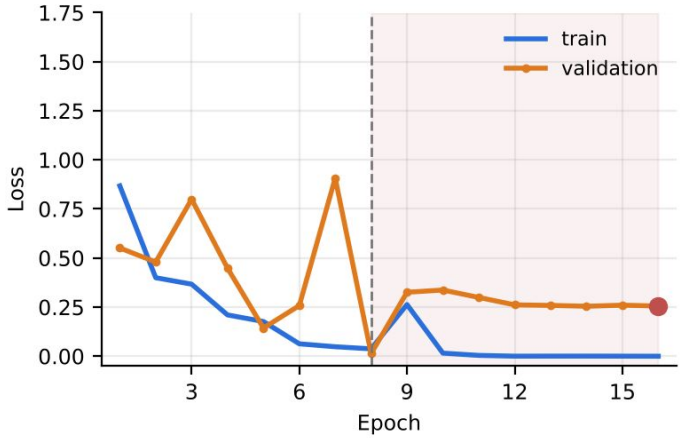
Why overconfident?

Training loss reached ≈ 0 and the test used the final, overfit epoch.

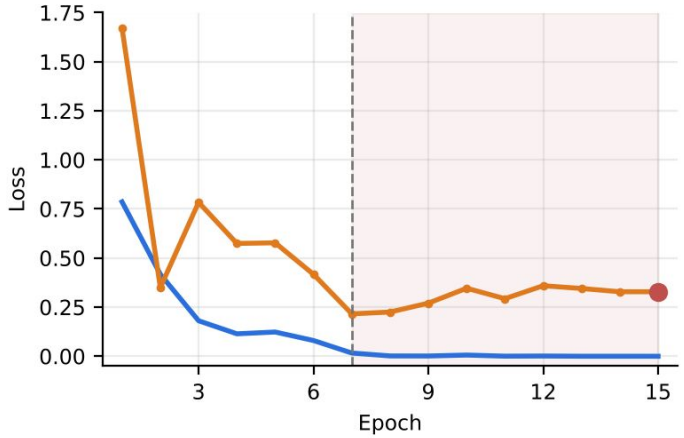
Results · training and validation loss per split

Blue = training · orange = validation · dashed = best validation epoch · red dot = epoch used for the test

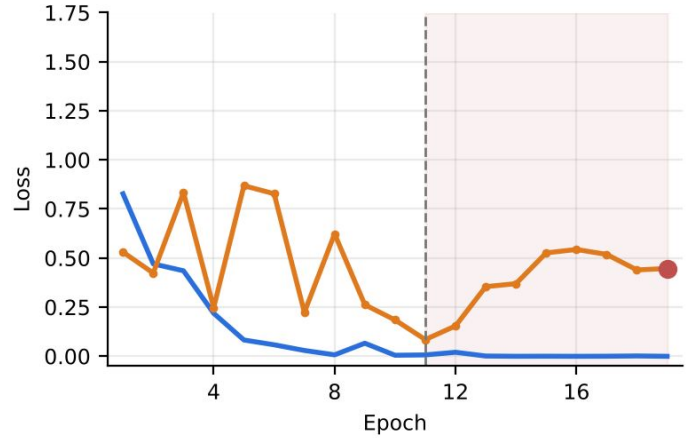
Split 01 · stopped at epoch 16
best epoch 8 · val loss 0.01 → 0.26



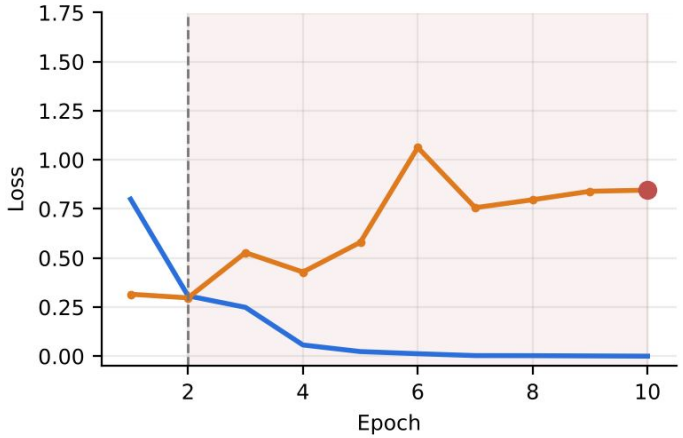
Split 02 · stopped at epoch 15
best epoch 7 · val loss 0.22 → 0.33



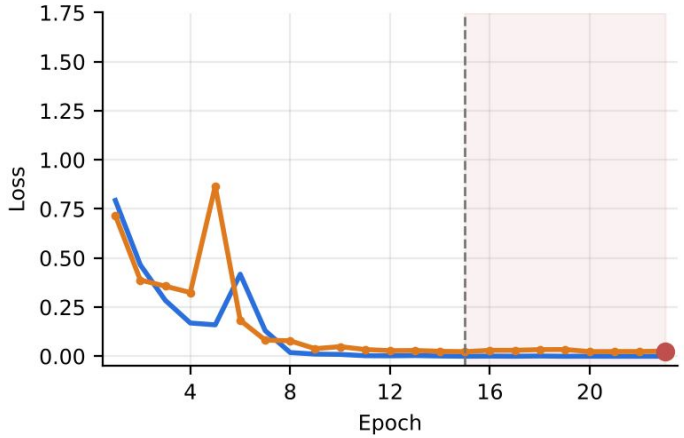
Split 03 · stopped at epoch 19
best epoch 11 · val loss 0.08 → 0.45



Split 04 · stopped at epoch 10
best epoch 2 · val loss 0.30 → 0.85



Split 05 · stopped at epoch 23
best epoch 15 · val loss 0.02 → 0.03



Training loss → about 0

The model memorizes its training cases (slightly negative = entropy bonus).

Validation loss bottoms out early

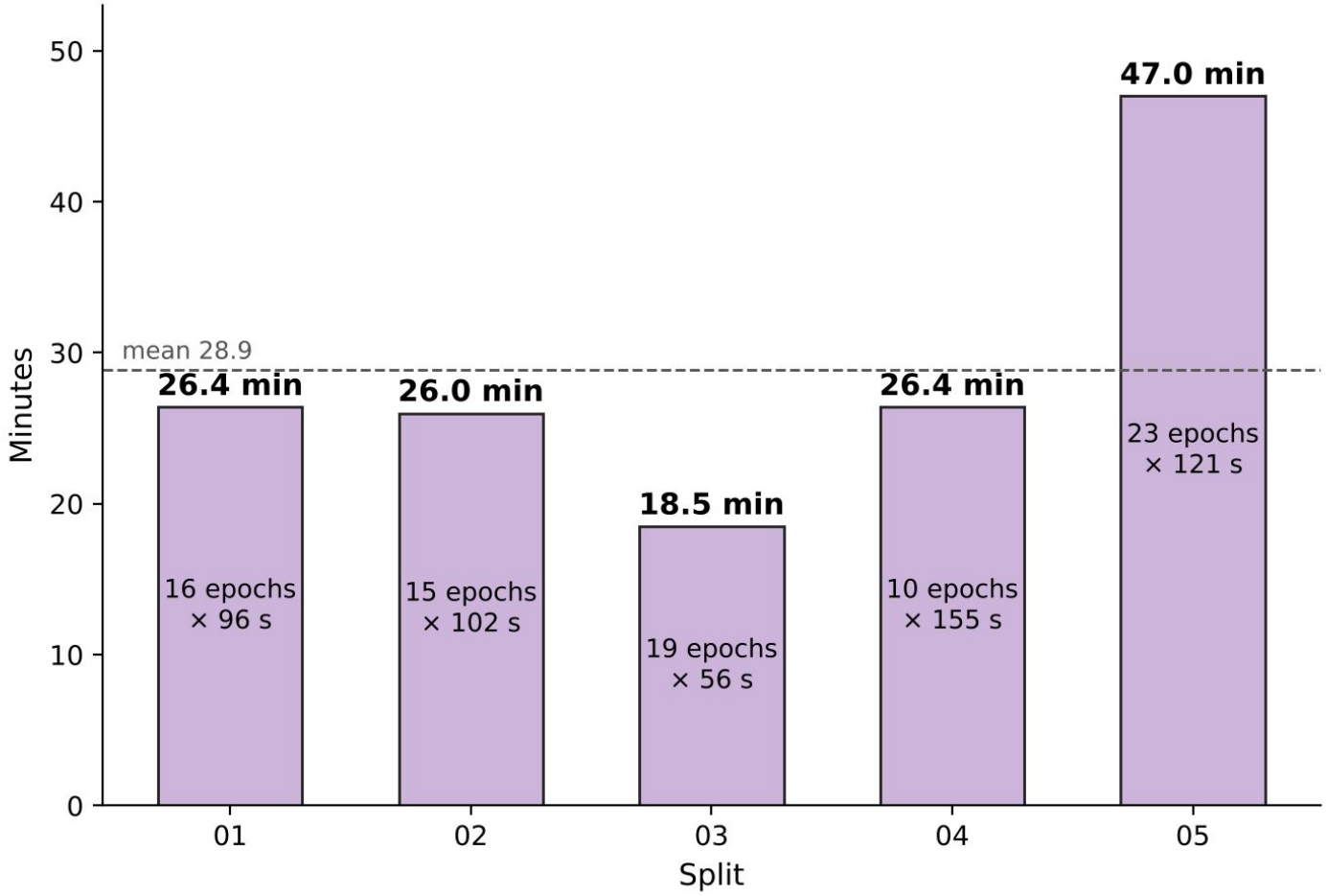
then rises or flattens: overfitting.

Tested 8 epochs past the best

Every split stopped at best + 8 (patience) and was tested with those final weights. Shaded = epochs after the best.

Results · runtime per split

Wall-clock time of main.py on Quest (16 cores, no GPU); the 5 splits ran in parallel



Runtime = epochs × seconds per epoch

Split 05: most epochs (23) at 121 s ⟶ 47.0 min

Split 03: 19 epochs at only 56 s ⟶ 18.5 min

Seconds per epoch varied 2.7× (56–155 s)

Same model, 55–57 training cases each. Compute is constant, so the variation points to reading feature files from shared storage (reflection).

Total: 144 min of compute (5 splits)

Every split stopped 8 epochs after its best (best epochs: 8, 7, 11, 2, 15).

Critical reflection · Is this the most efficient code?

Short answer: well organized, but not efficient to run. Evidence from the code and from my 5 runs

Organization

Mostly good

+ Clean modules: config · data · dataset · model · trainer · main

+ Splits saved as .npz → reproducible

− Model class copied into optuna_training.py (tuned 2 dropouts, trains with 1)

− Paths hard-coded in 4 files; README lists flags that don't exist

− --seed in make_splits.py was ignored (fixed for this project)

Operations

Suggested Changes

− Test uses the LAST epoch, not the best: all 5 splits tested 8 epochs past best

split 04: val loss 0.30 (best) → 0.85 (tested)

− Saves a ~30 MB checkpoint every epoch (up to ~0.9 GB per split)

Fix: Load best.pth before evaluate() (2 lines in main.py)

− Class weights [2.53, 1] do nothing: one case per batch, so the weight cancels

Fix: reduction="sum" in trainer.py, then re-tune the weight

Execution time

I/O-bound

− Every epoch re-opens up to 7,500 tiny .pt files per case (torch.load each)

− Measured per full-size case: 2.15 s loading vs 0.37 s model compute

− My runs: 56–155 s per epoch for the same model and data → time isn't the model

Verdict: not the most efficient. The modules are clean, but most of the runtime is spent re-reading files, and the reported test results come from overfit models, and the class weights have no effect.

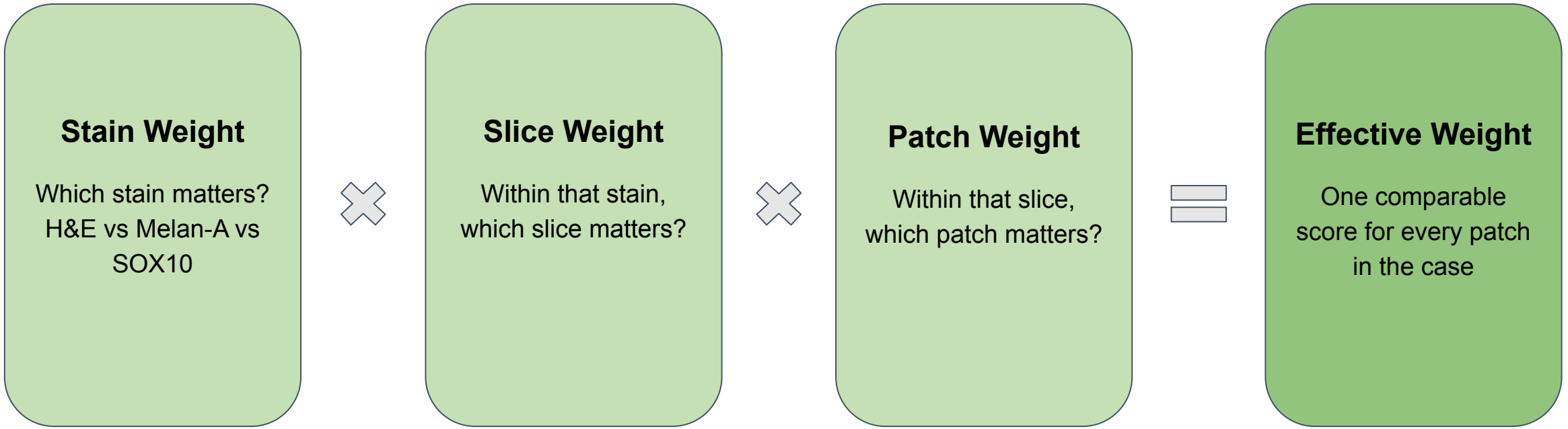
Analysts

attention_analysis.py – How does it work?

	1	2	3	4	5	6
Per Slice	**Run model** Predict, keep the attention weights, save the true and predicted label function analyze_attention_weights	**Not used** This path ranks by the raw patch weight W	**Rank by W** Best and worst slice of each stain function analyze_case_attention np.argsort	**Find image** (stain, slice #, patch #) becomes a PNG file function _lookup_png_for_patch	**Per-slice grid** Top / bottom N. Title shows W function visualize_patch_attention	**Summarize** After the loop: 4 outputs from the collected records function save_attention_summary plot_effective_patch_attention_distribution_per_case plot_slice_attention_distribution_per_caseandstain analyze_top_effective_patches_per_case
Per Case		**Effective weight** stain × slice × patch function compute_effective_patch_attention	**Rank by Eff** All patches in the case together function sorted		**Per-case grid** Top / bottom N. Title shows Eff function visualize_case_effective_patches	

attention_analysis.py - Focusing on Step 2 - Scoring the Patches

Three levels of attention multiplied into one comparable effective weight.



attention_analysis.py - Focusing on Step 4 - Finding the Picture

We take the position and find the picture. The model doesn't see the filenames, so the code rebuilds the file list and counts to the position.

Model Gives a Position

Stain, slice, patch

function records from compute_effective_patch_attention,

List Files

Search for case_<id>_*.pt, also the zero-padded id

function _build_pt_index_for_case_stain

Keep One Stain

Filter filenames for h&e, melan or sox10

function _file_contains_stain

Group by Slice

{match_1: […], unmatched_6: […]}

function _parse_slice_id_from_filename

Sort & Count

Sort slice names; take slice #. Sort patches by number; take patch # *patch0 sorts last (0 or 10**18)

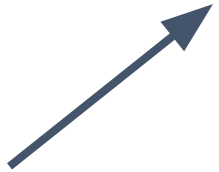
function _parse_patch_index_from_pt_filename

Open PNG File

Swap .pt for .png in the picture folder and load it

function _map_pt_to_png_path _lookup_png_for_patch

If anything fails, the code will silently fall back to a PNG search (_build_png_index_for_case_stain), then to a raw-number heatmap (_plot_feature_heatmap)



attention_analysis.py - Focusing on Step 4 - Risk

Is the picture the actual patch?

The model saw: text sort in data_utils / dataset
But in attention_analysis.py, the lookup shows: number sort in attention_analysis.

data_utils.py builds the model's list

Lines 49-50

```
# Deterministic ordering (important for attention patch_idx -> filename mapping)
all_files = sorted(os.listdir(patches_dir))
```

Lines 91-93

```
# Ensure deterministic ordering of patch paths WITHIN each (case_id, slice_id)
for k in list(case_slices.keys()):
   case_slices[k] = sorted(case_slices[k])
```

Line 219-221

```
# Store (slice_id, sorted_patch_list) so slices can be sorted later
for stain in sorted(stain_groups.keys()):
   stain_patches = sorted(stain_groups[stain])
```

Plain sorted() compares text, character by character: **patch10 comes before patch2.**

attention_analysis.py step 4 · the lookup's list

Lines 270-273

```
def _parse_patch_index_from_pt_filename(fname: str) -> Optional[int]:
    base = os.path.basename(fname)
    m = re.search(r"_patch(\d+)\.pt$", base, flags=re.IGNORECASE)
    return int(m.group(1)) if m else None
```

Lines 320-326

```
flist_sorted = sorted(flist, key=lambda x: (
    _parse_patch_index_from_pt_filename(x) is None,
    _parse_patch_index_from_pt_filename(x) or 10**18,
    x,
))
```

Sorts by number, but Python treats 0 as false, so **0 or 10**18 gives 10**18 and patch0 goes last.**

attention_analysis.py - Focusing on Step 5 - Visualizing

Build two visualizations, both built from the same ranked positions and the same image lookup.

Ranked positions

Top N and bottom N, with their weights

function
np.argsort
sorted

A. Per slice

Top N patches of the best slice, bottom N of the worst slice (ranked by W)

function
analyze_case_attention → visualize_patch_attention

B. Per case

All patches ranked together. Title shows stain, slice, patch and effective weight

function
visualize_case_effective_patches

Visualizations

Tissue patch found by the lookup + its attention value

function
patch_attention/
case_effective_patches/

attention_analysis.py - Focusing on Step 6 - Summarizing

During the loop, three lists collect data for each test case: all_patch_records, case_label_info (true vs. predicted labels), and slice_attention_records.

After the loop, four functions save the collected data to files.

Stain Summary

attention_summary.txt: stain weights per case

function save_attention_summary

Patch Histograms

How attention is spread over a case. A spike means the model focuses on few patches

function plot_effective_patch_attention_distribution_per_case

Slice Rank Plots

Ranked slice weights vs. a dashed “equal attention” line

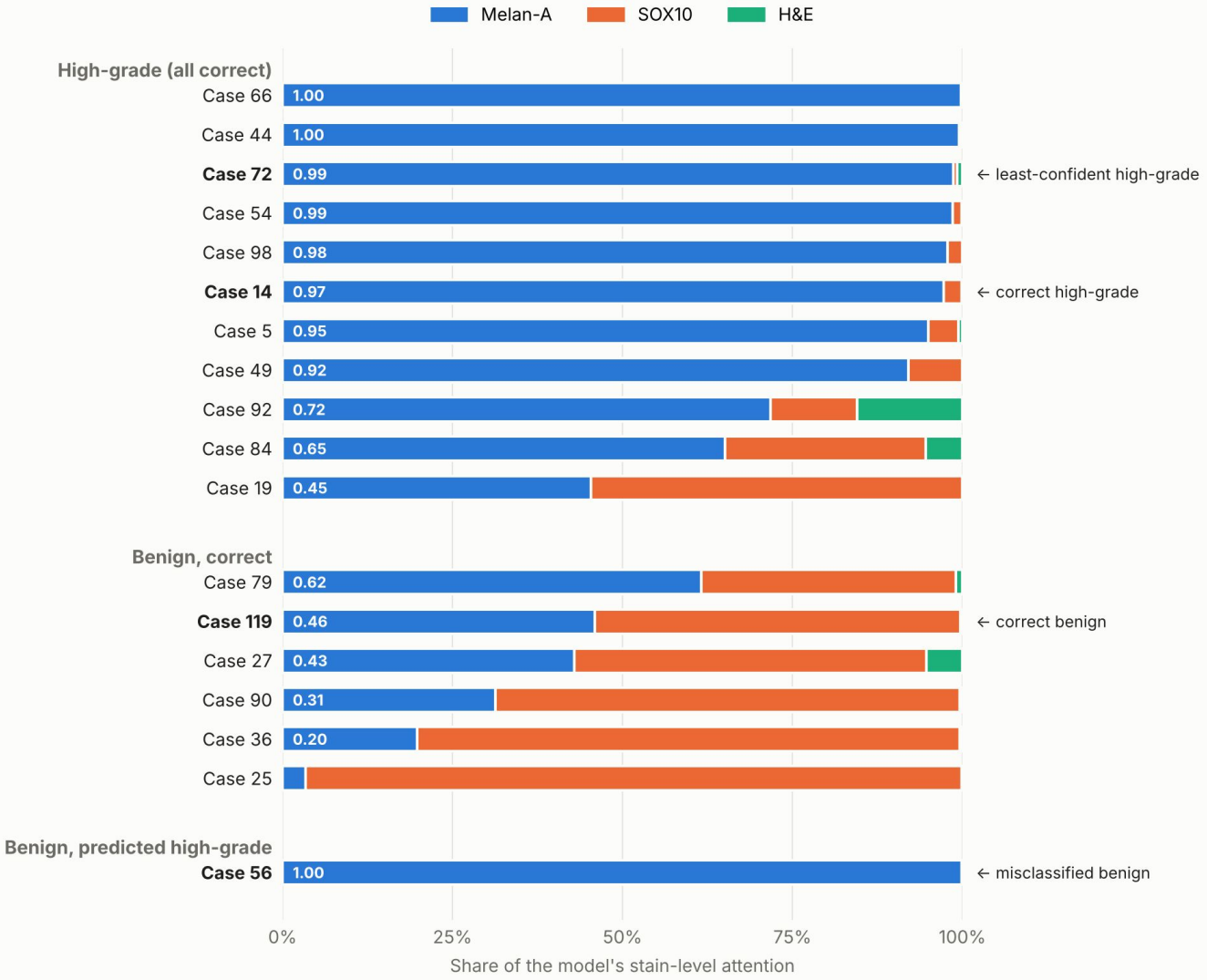
function plot_slice_attention_distribution_per_caseandstain

Top 5% Table

CSV of the top patches, with counts by stain and slice

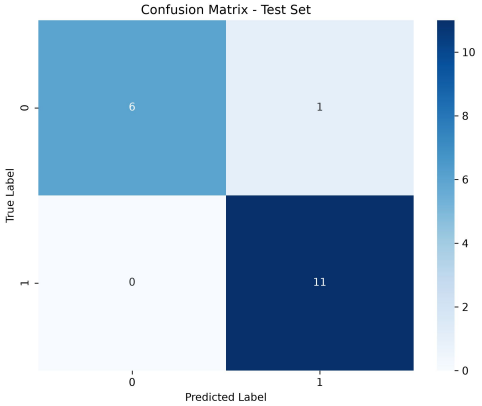
function analyze_top_effective_patches_per_case

Results · Where does the model look? (split 1, 18 test cases)



Melan-A has the top stain weight in 10 of 11 high-grade cases; SOX10 in 5 of 6 correct benign cases. H&E is at or below 0.06 in 17 of 18 cases. Case 56 (benign, misclassified) puts 0.999 of its stain weight on Melan-A.

Test predictions: 17 of 18 correct



Rows = true, columns = predicted. One false alarm (case 56), no missed high-grade. Sources: flowchart step 1 (predictions) and step 6 (attention_summary.txt).

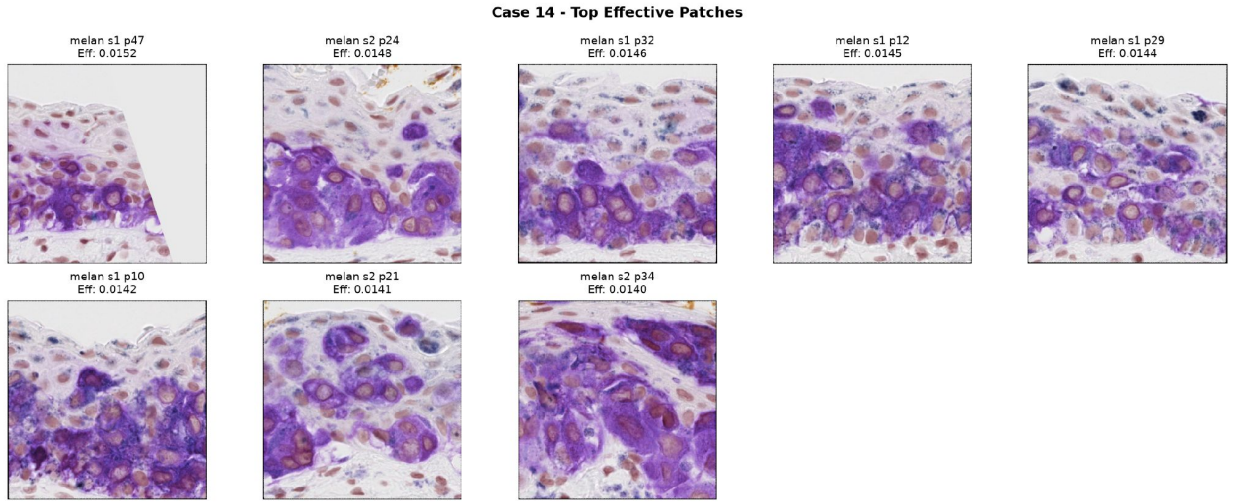
0.87 vs 0.34 — average Melan-A weight: high-grade cases vs correctly classified benign cases

≤ 0.06 — H&E weight in 17 of 18 cases (max 0.15, case 92)

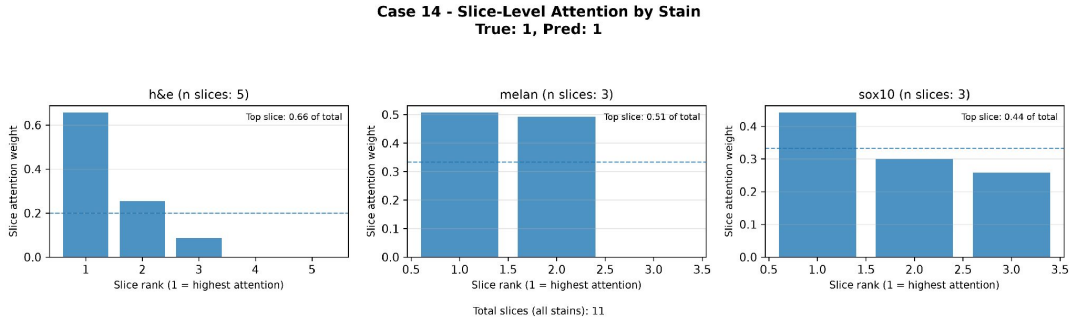
1–2 slices — hold all of a case's top-5% patches in 14 of 18 cases

Case 14 · correctly classified high-grade (P = 1.00)

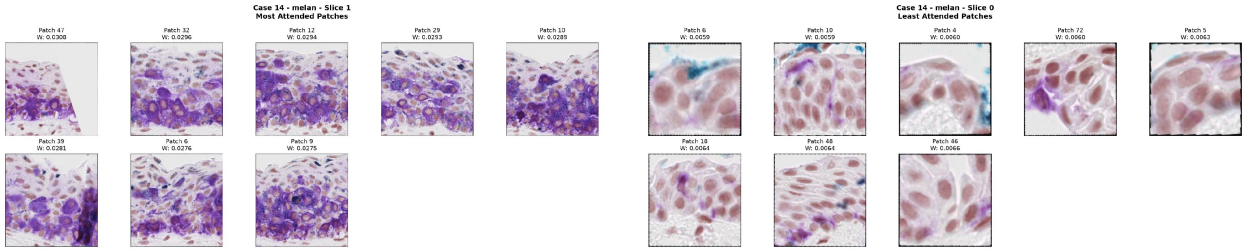
Per case · top 8 patches by effective weight (Eff)



Per stain · slice weights (dashed = equal attention)

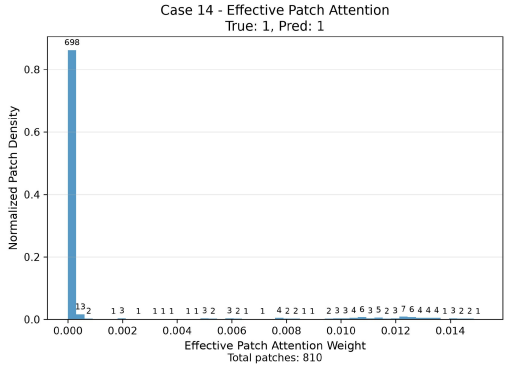


Per slice · Melan-A, most vs least attended slice (patch weight W)



Slice 1 · most attended · W 0.028–0.031

Slice 0 · least attended · W 0.006–0.007



Most patches get near-zero weight

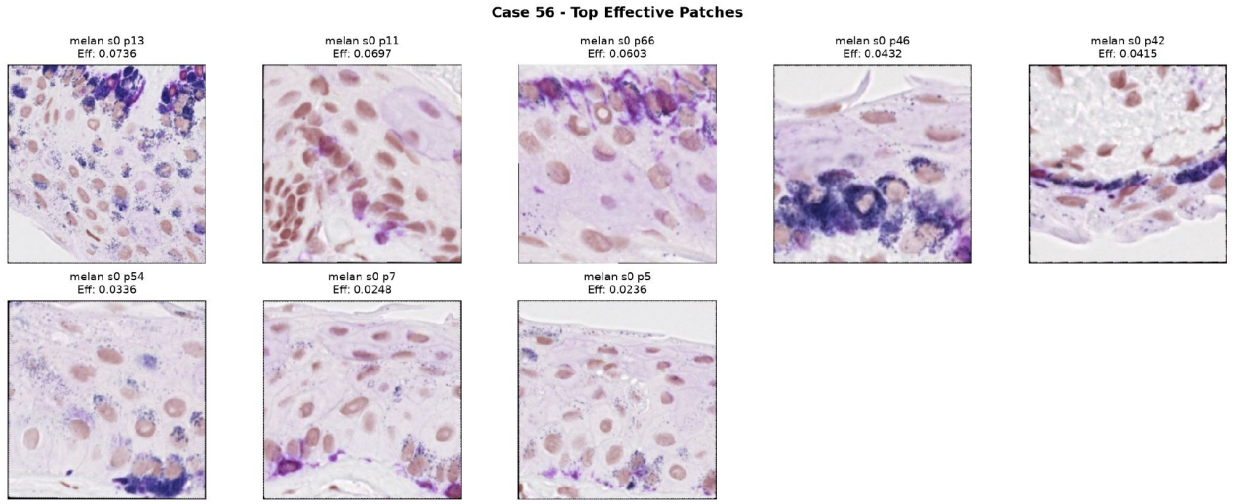
698 of 810 patches fall in the first histogram bin (Eff ≈ 0).

In numbers Melan-A has 0.97 of the stain weight, split 0.51 / 0.49 between two slices, and all 40 top-5% patches come from those two slices.

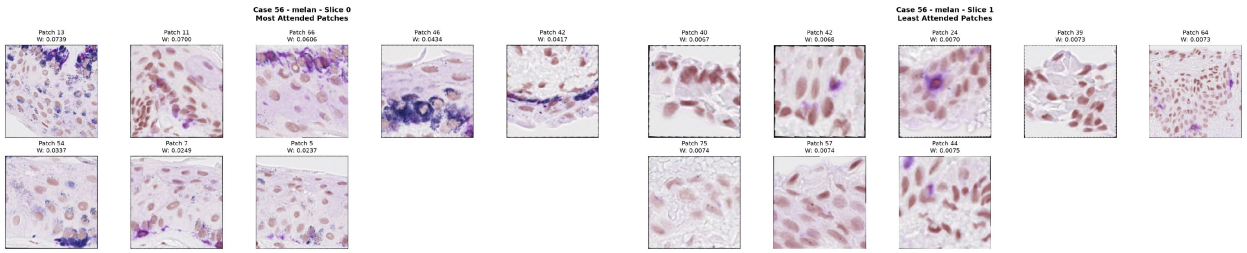
Flowchart: tiles = steps 2→3→4→5 · slice plot, histogram and stain weights = step 6. Numbers come from the model; tiles depend on step 4 and may show a different patch from the same slice.

Case 56 · misclassified benign (true benign, P(high) = 0.91)

Per case · top 8 patches by effective weight (Eff)



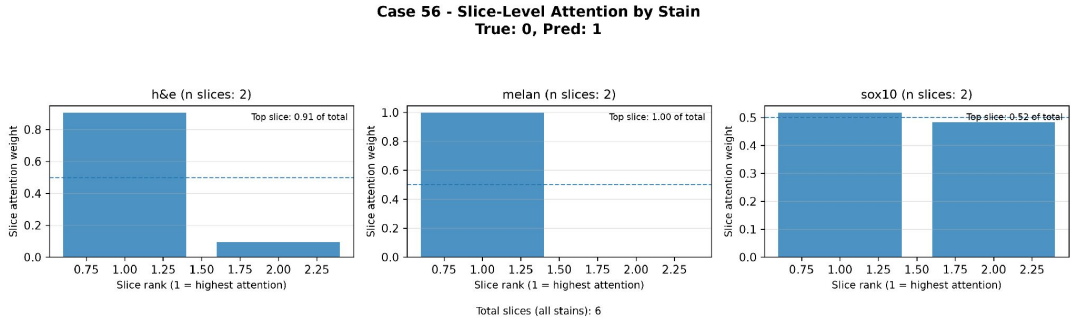
Per slice · Melan-A, most vs least attended slice (patch weight W)

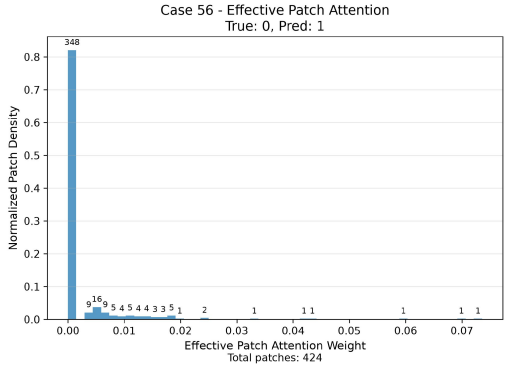


Slice 0 · slice weight 1.00 · W up to 0.074

Slice 1 · slice weight ≈ 0 · W 0.007

Per stain · Melan-A 1.00 · SOX10 0.00 · H&E 0.00





Attention is concentrated

Top 3 patches = 20.4% of the effective weight

Case 14: 4.5% · case 119: 4.4%

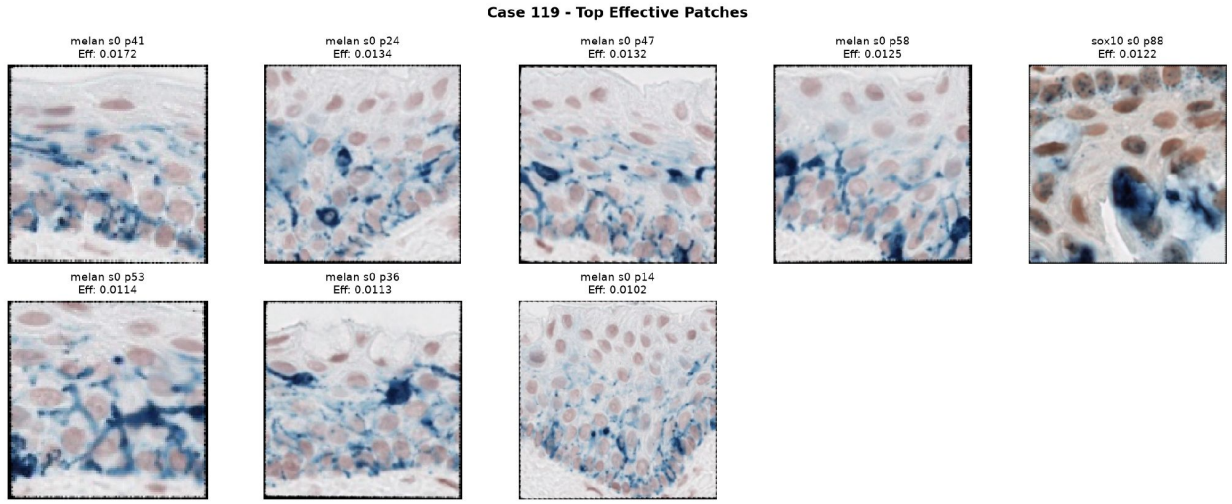
Top 8 = 37.0% (case 14: 11.6%)

In numbers One Melan-A slice gets almost all of Melan-A's weight, and the top 3 patches hold 20.4% of the effective weight (case 14: 4.5%). Whether this caused the error has not been tested.

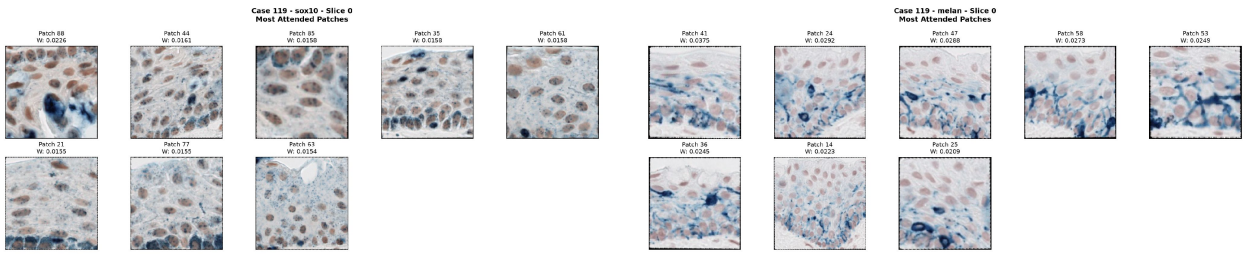
Flowchart: tiles = steps 2→3→4→5 · slice plot, histogram and stain weights = step 6. Numbers come from the model; tiles depend on step 4 and may show a different patch from the same slice.

Case 119 · correctly classified benign (P(high) < 0.001)

Per case · top 8 patches by Eff: 7 of 8 are Melan-A



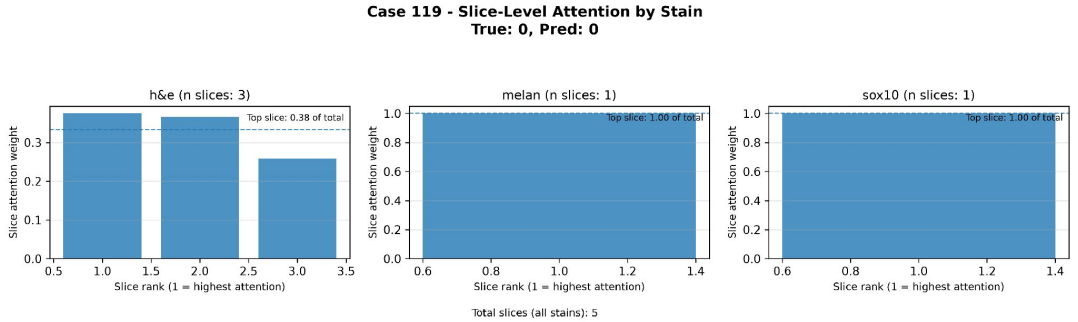
Per slice · most attended patches, SOX10 vs Melan-A (one slice each)



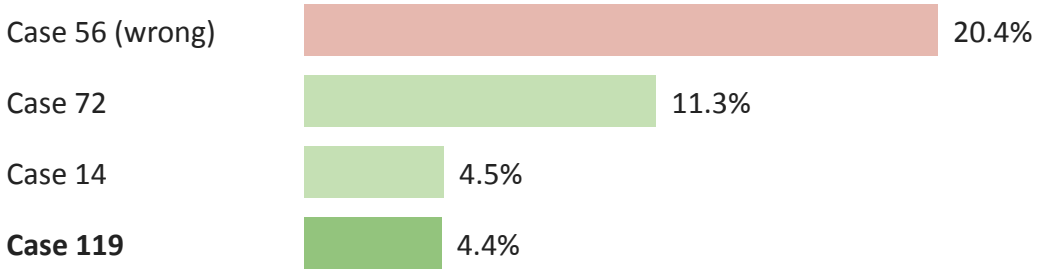
SOX10 · stain weight 0.54 · W 0.015–0.023

Melan-A · stain weight 0.46 · W 0.021–0.038

Per stain · SOX10 0.54 · Melan-A 0.46 · H&E 0.00



Share of effective weight held by the top 3 patches



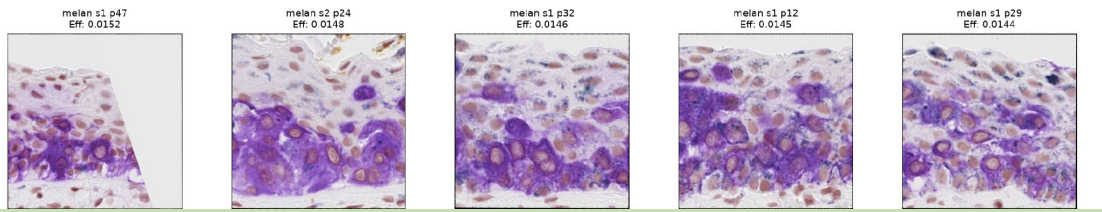
In numbers Weight is spread over two stains; the top 3 patches hold 4.4%. SOX10 has the higher stain weight, yet Melan-A patches have higher patch weights and fill 7 of the top 8. Patch counts were not checked.

Flowchart: tiles = steps 2⟶3⟶4⟶5 · slice plot, histogram and stain weights = step 6. Numbers come from the model; tiles depend on step 4 and may show a different patch from the same slice.

The four cases · top 5 patches as displayed by the code

Case 14 Correct high-grade

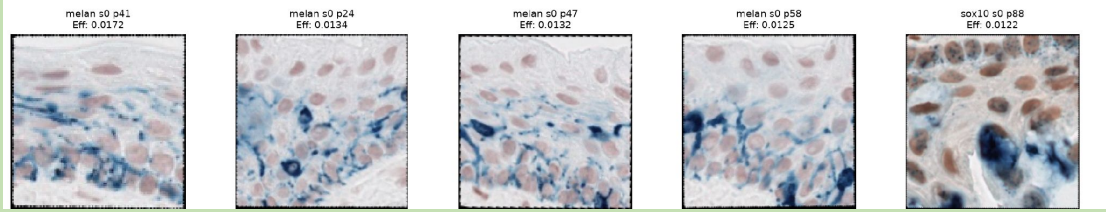
P(high) = 1.00 · Melan-A 0.97 · SOX10 0.03



- All 40 top-5% patches from Melan-A slices 1–2
- Top 8 effective weights all between 0.014 and 0.015

Case 119 Correct benign

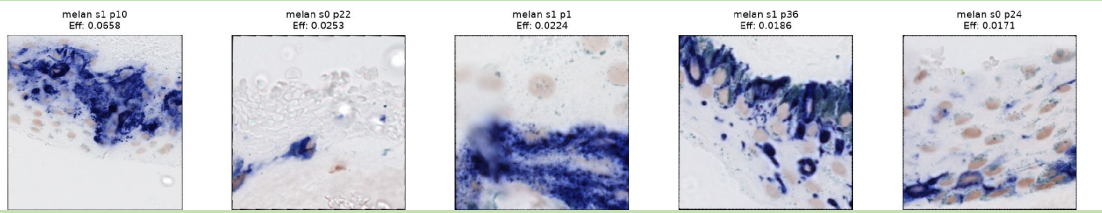
P(high) < 0.001 · SOX10 0.54 · Melan-A 0.46



- 15 Melan-A + 9 SOX10 top patches: balanced
- Melan-A holds more top patches despite lower stain weight

Case 72 No false negative → closest call*

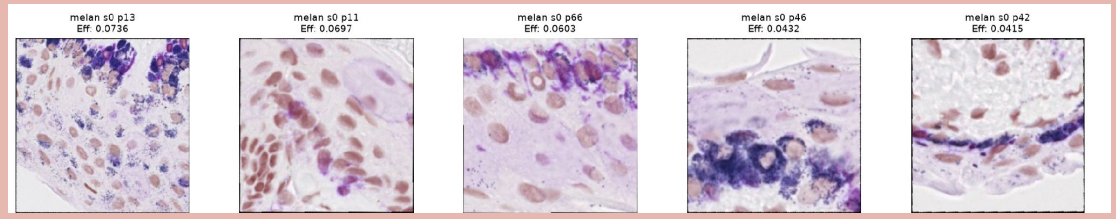
P(high) = 0.65 · Melan-A 0.99 · 0 of 11 high-grade missed



- Top patch (s1 p10) Eff 0.066; next highest 0.025
- Close calls: 84 (0.66), 44 (0.84). 11 cases: miss rate up to ~28%

Case 56 Misclassified benign

P(high) = 0.91 (wrong) · Melan-A 1.00



- All 21 top patches from one Melan-A slice (of 6)
- Top 8 patches hold 37.0% of the effective weight

*Flowchart: tiles = steps 2→3→4→5. Numbers come from the model; tiles depend on step 4 and may show a different patch from the same slice. * Split 1 had no misclassified high-grade case; case 72, the least confident correct call, stands in (our seed-1805 runs missed 8).*

Critical reflection · How well does attention_analysis.py work?

What seems to work

1 **Step 1 · Run model**

Weights come straight from the trained model (return_attn_weights=True), not from an approximation.

2 **Step 1 · Run model**

No randomness at test time (model.eval(), shuffle_patches=False), so re-runs should give the same weights.

3 **Step 2 · Score patches**

Effective weight = stain × slice × patch. Each level sums to 1, so a case's patches sum to 1 and can be compared.

Where it falls short

4 **Step 4 · Find image** _lookup_png_for_patch

The model knows a patch only by its position, e.g. #13. The lookup rebuilds the file list by sorting the folder by number, and 0 or 10**18 sends patch0 to the end; data_utils sorted the model's list as text (slide 23). When the two orders differ, #13 opens a different file, and no error is raised.

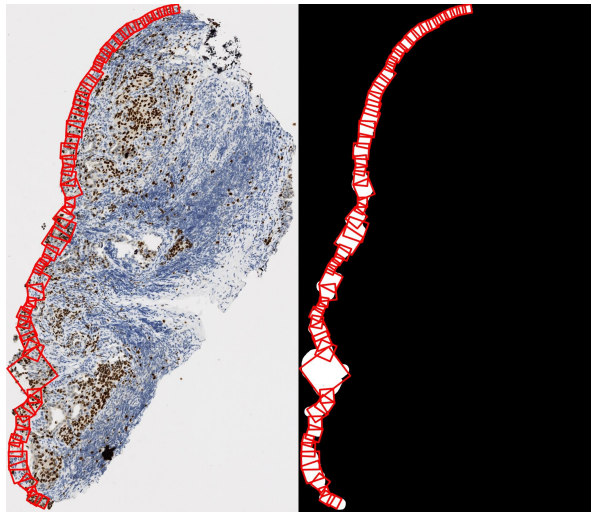
Effect on our results: the weights and the stain and slice findings stand; the tissue in the tiles can't be trusted yet.

Next step: pass the filenames the model actually received (from dataset.py) to the lookup, then re-run with the saved model.

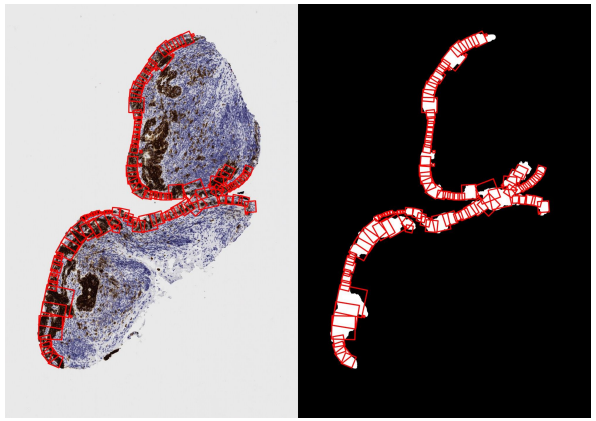
5 **In General** Split 1 is not generalizable. Too small of test set.

Data Engineering

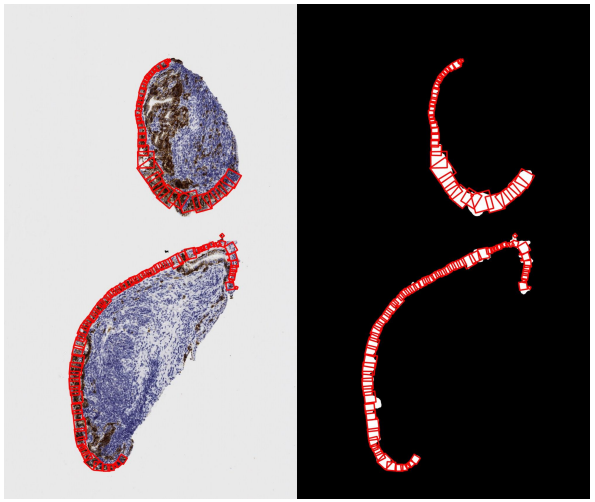
Case 8



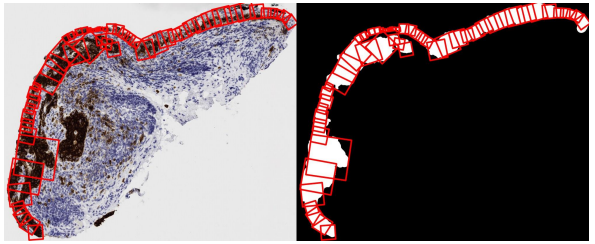
SOX10 · Match 1



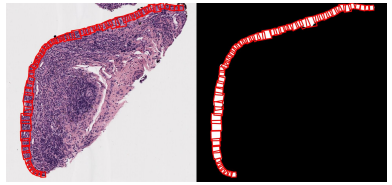
Melan · Unmatched 1



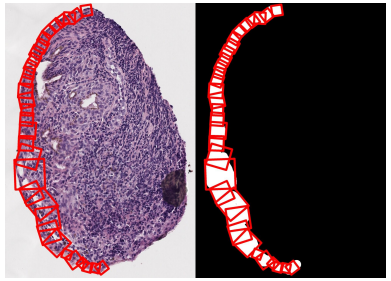
Melan · Unmatched 2



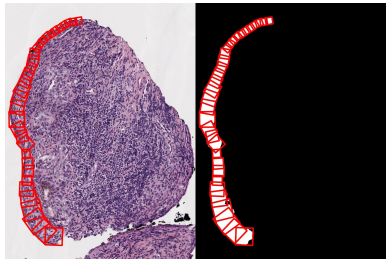
Melan · Match 1



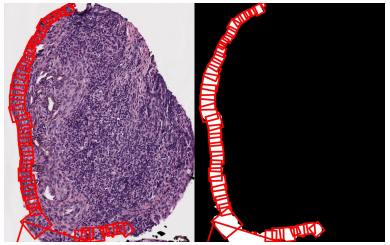
H&E · Match 1



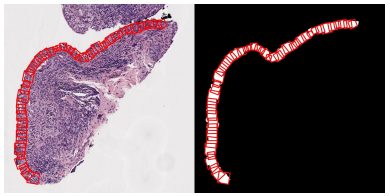
H&E · Unmatched 1



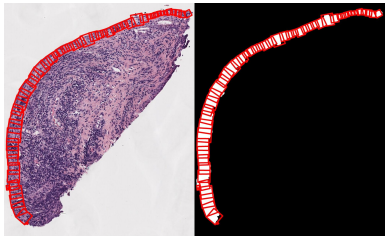
H&E · Unmatched 2



H&E · Unmatched 3

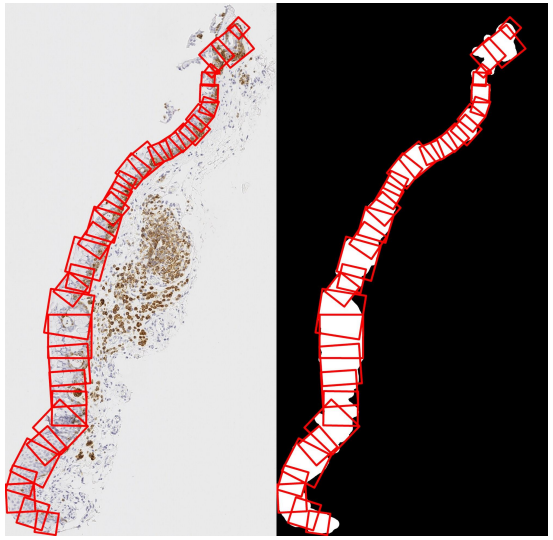


H&E · Unmatched 4

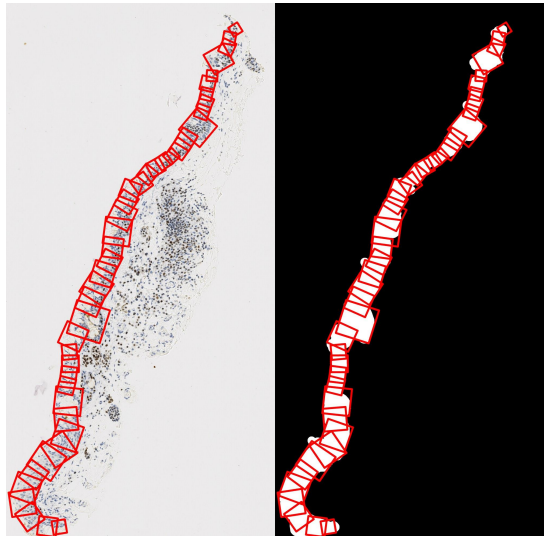


H&E · Unmatched 5

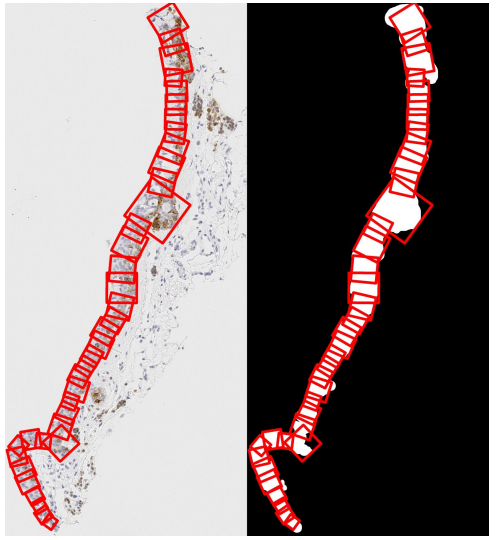
Case 20



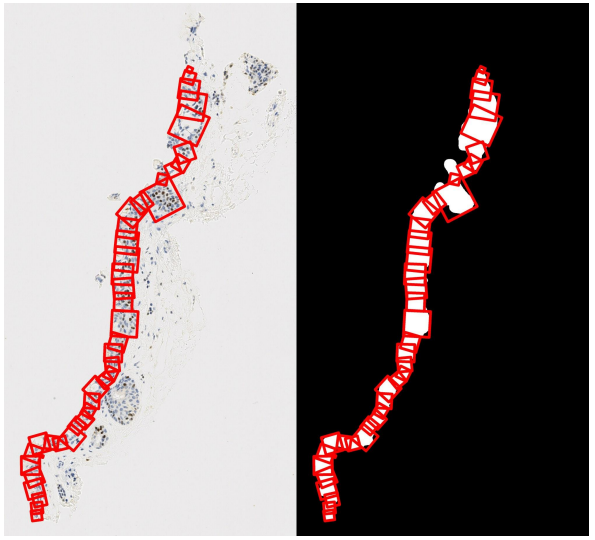
Melan · Match 1



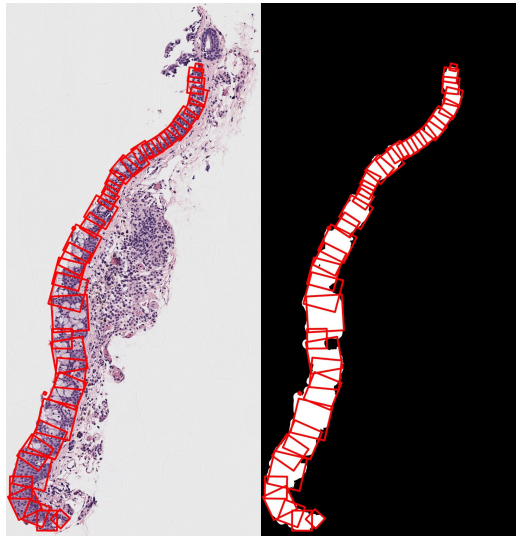
Sox10 · Match 1



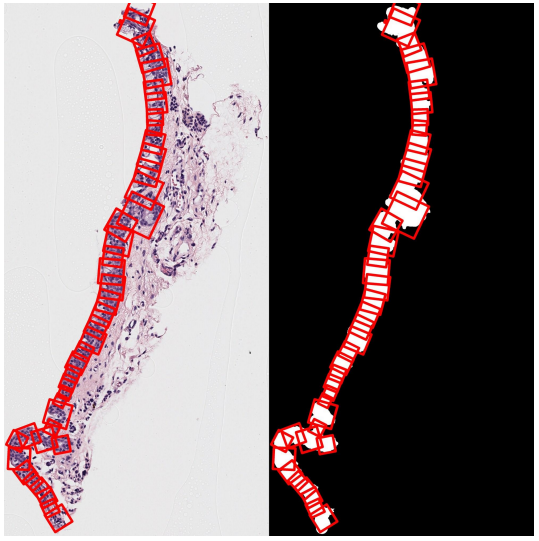
Melan · Match 2



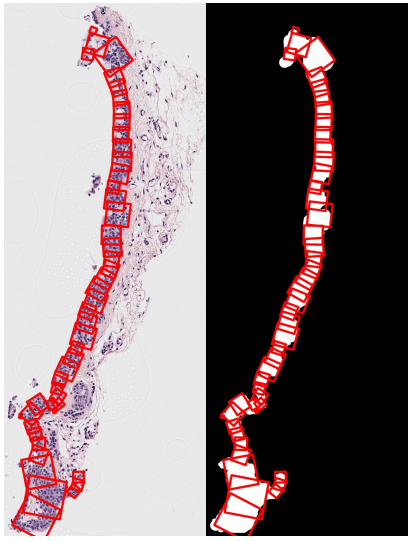
Sox10 · Match 2



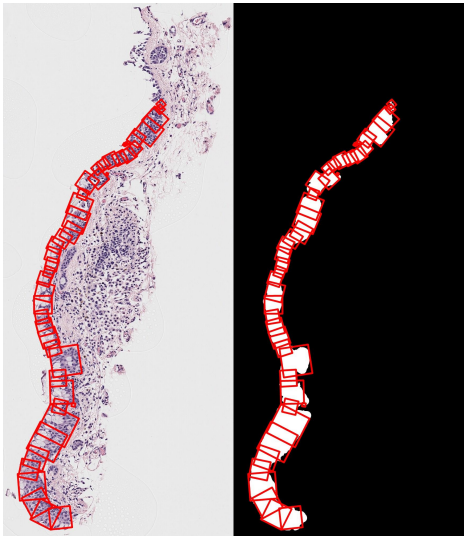
H&E · Match 1



H&E · Match 2

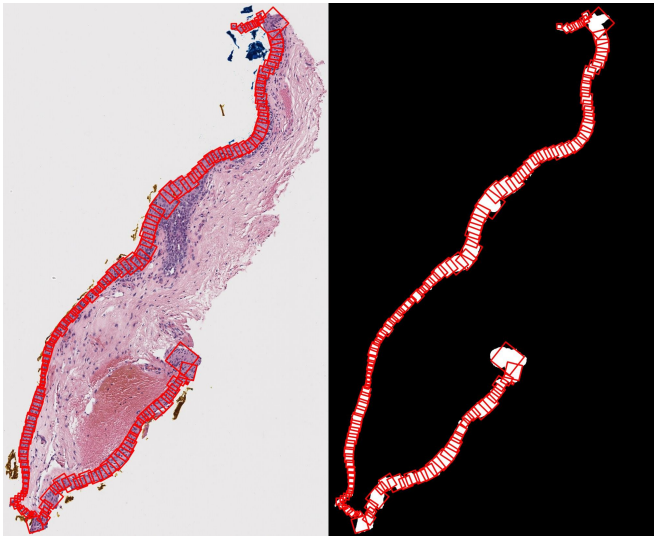


H&E · Unmatched 1

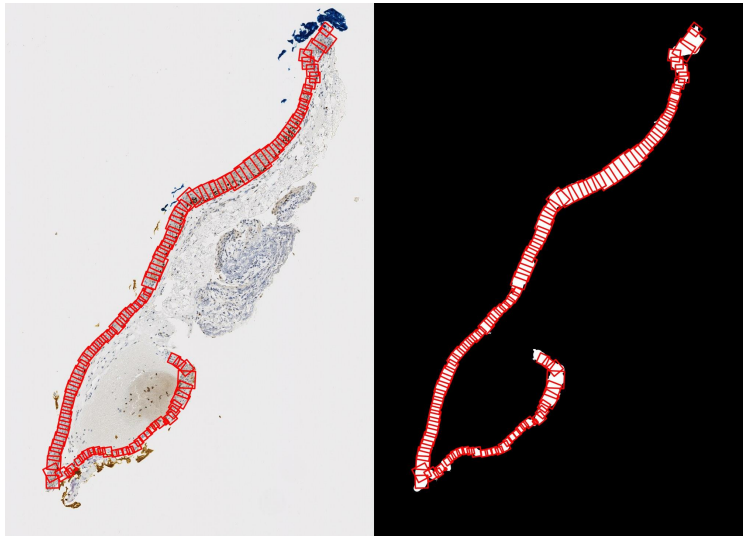


H&E · Unmatched 2

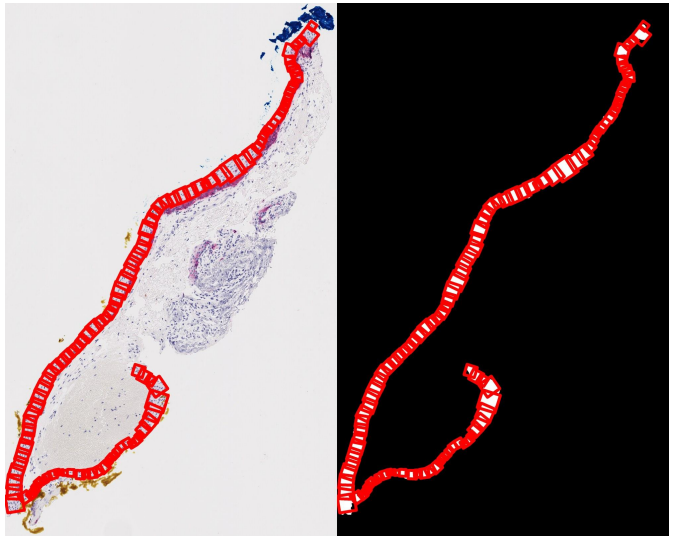
Case 33



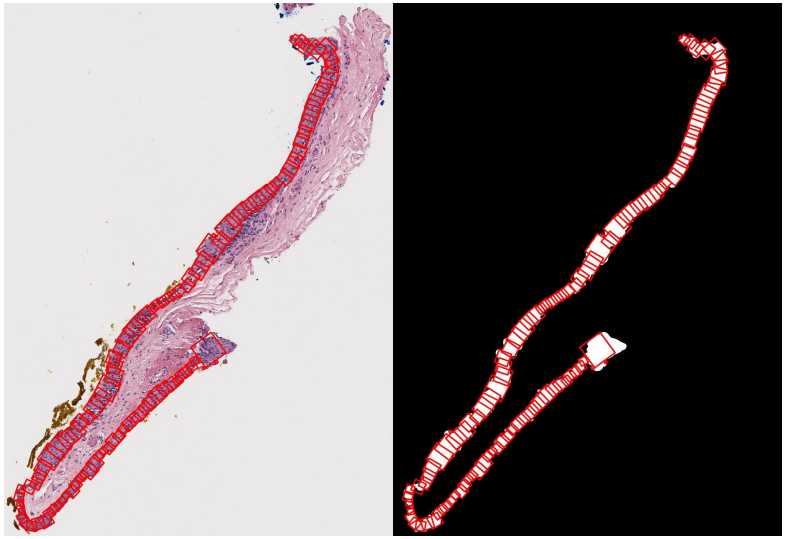
H&E · Match 1



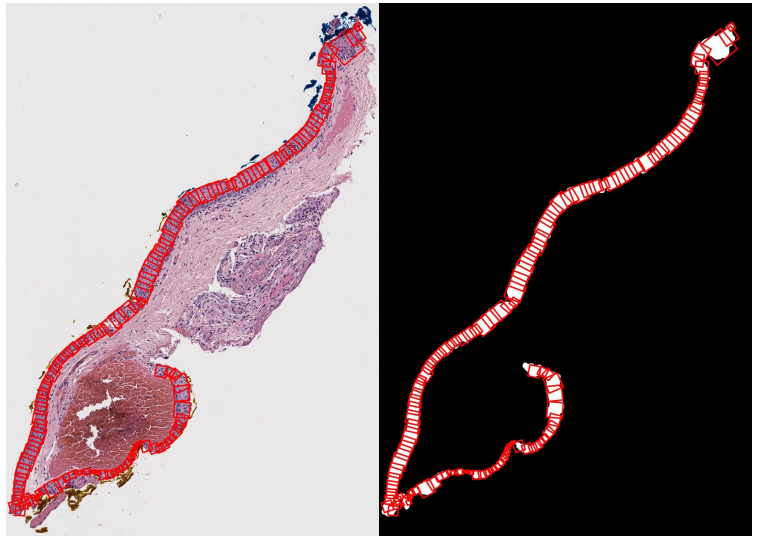
SOX10 · Match 1



Melan · Match 1

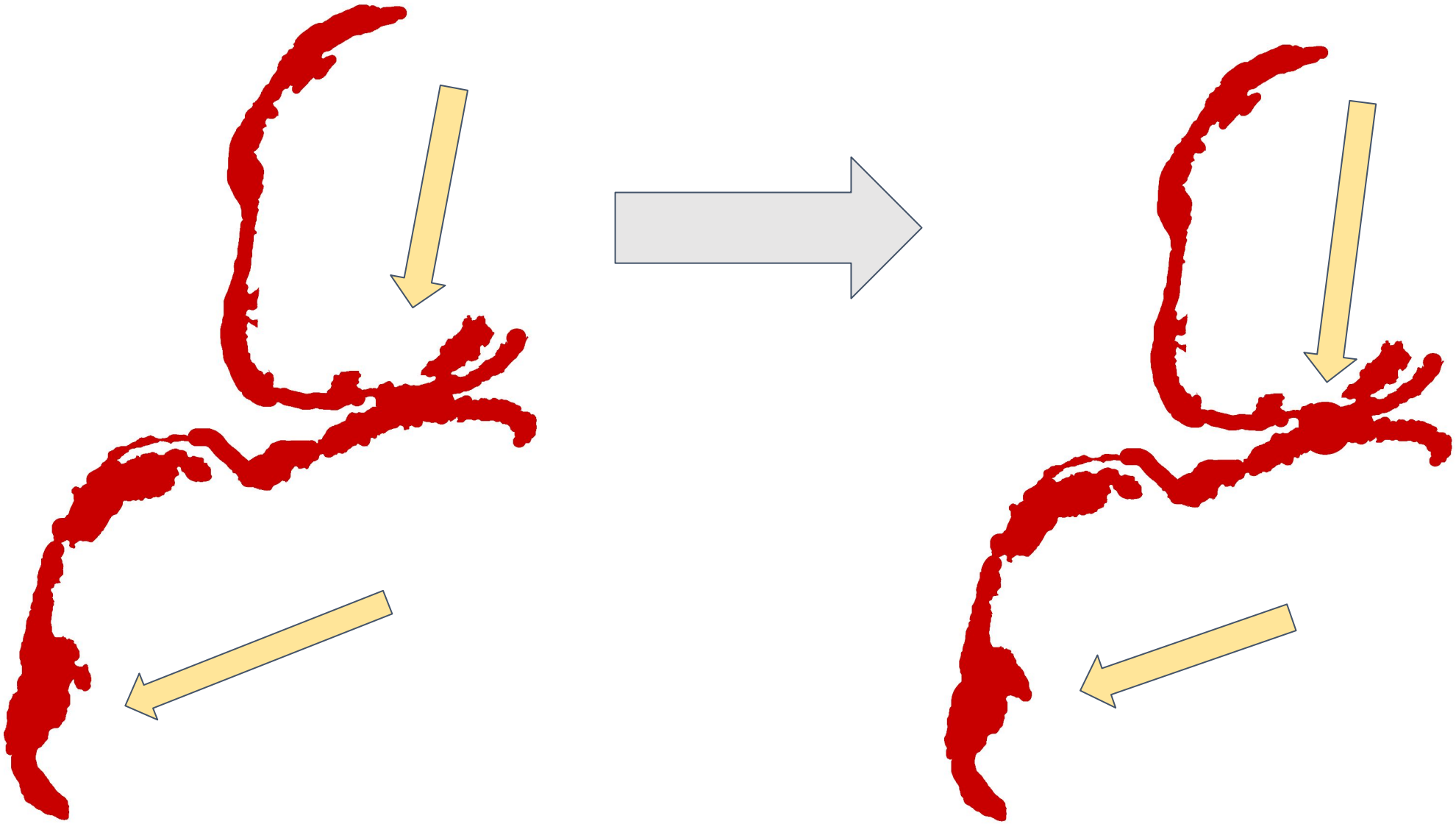


H&E · Unmatched 1

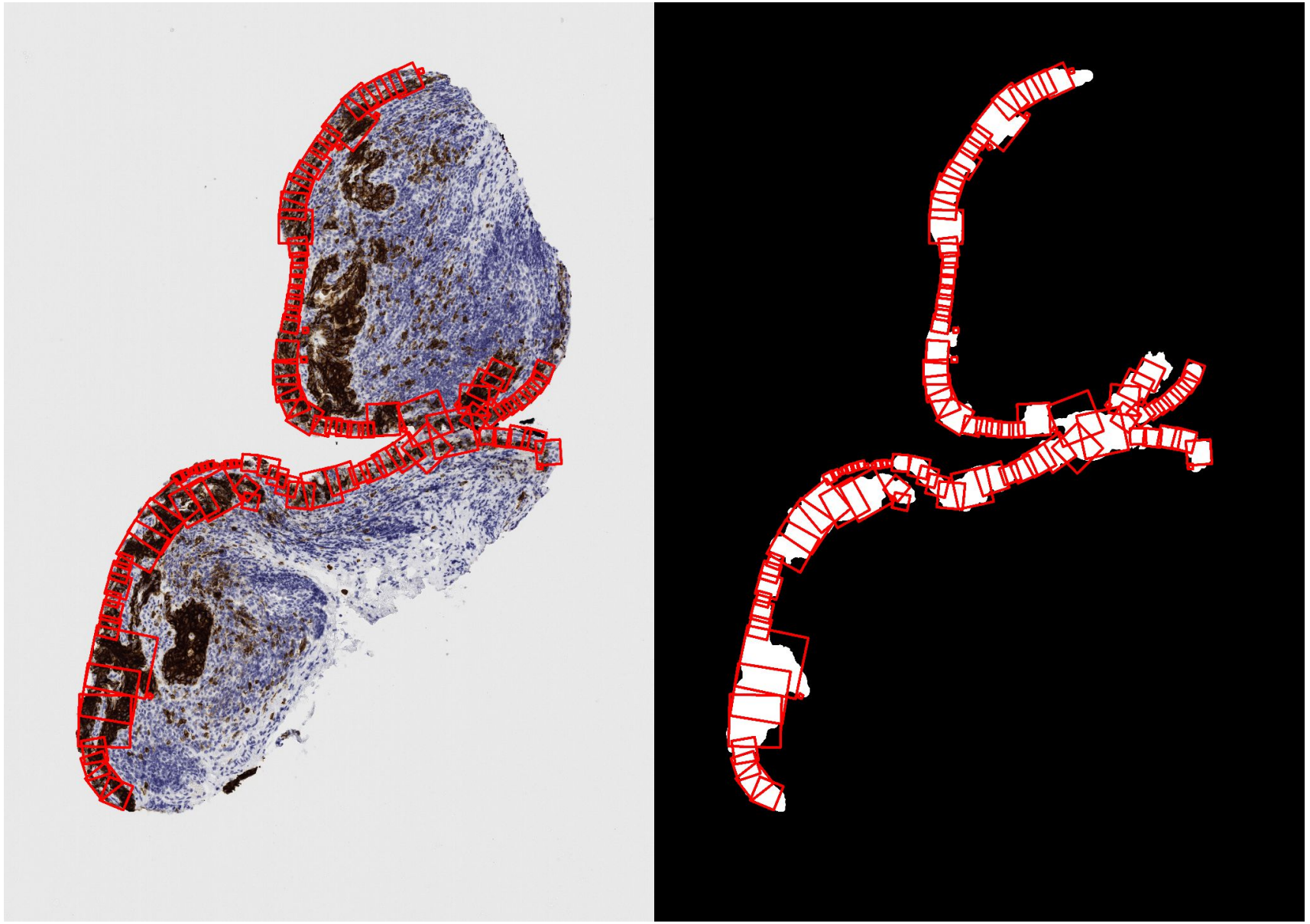


H&E · Unmatched 2

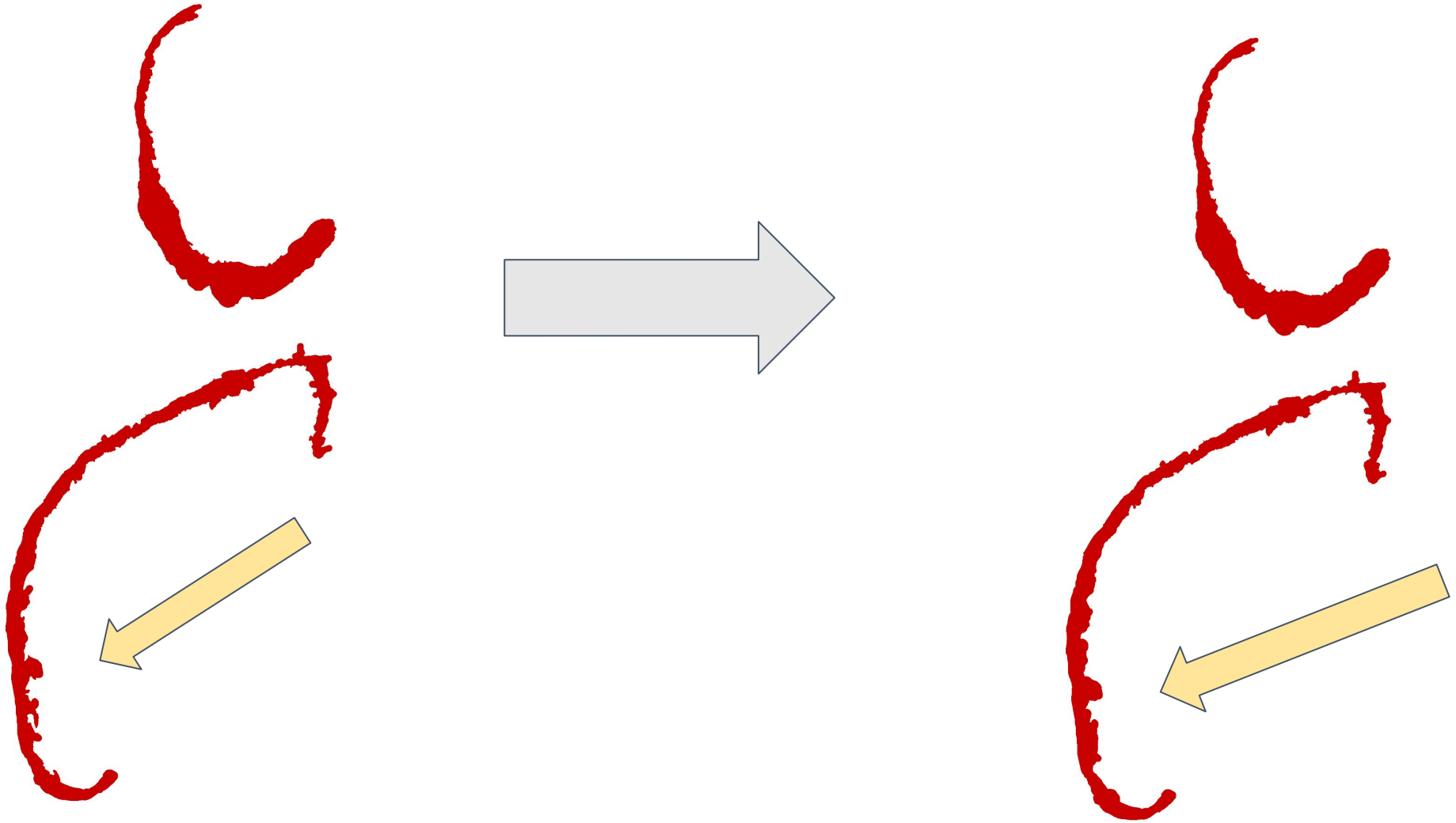
Case 8, Unmatched 1, Melan Stain



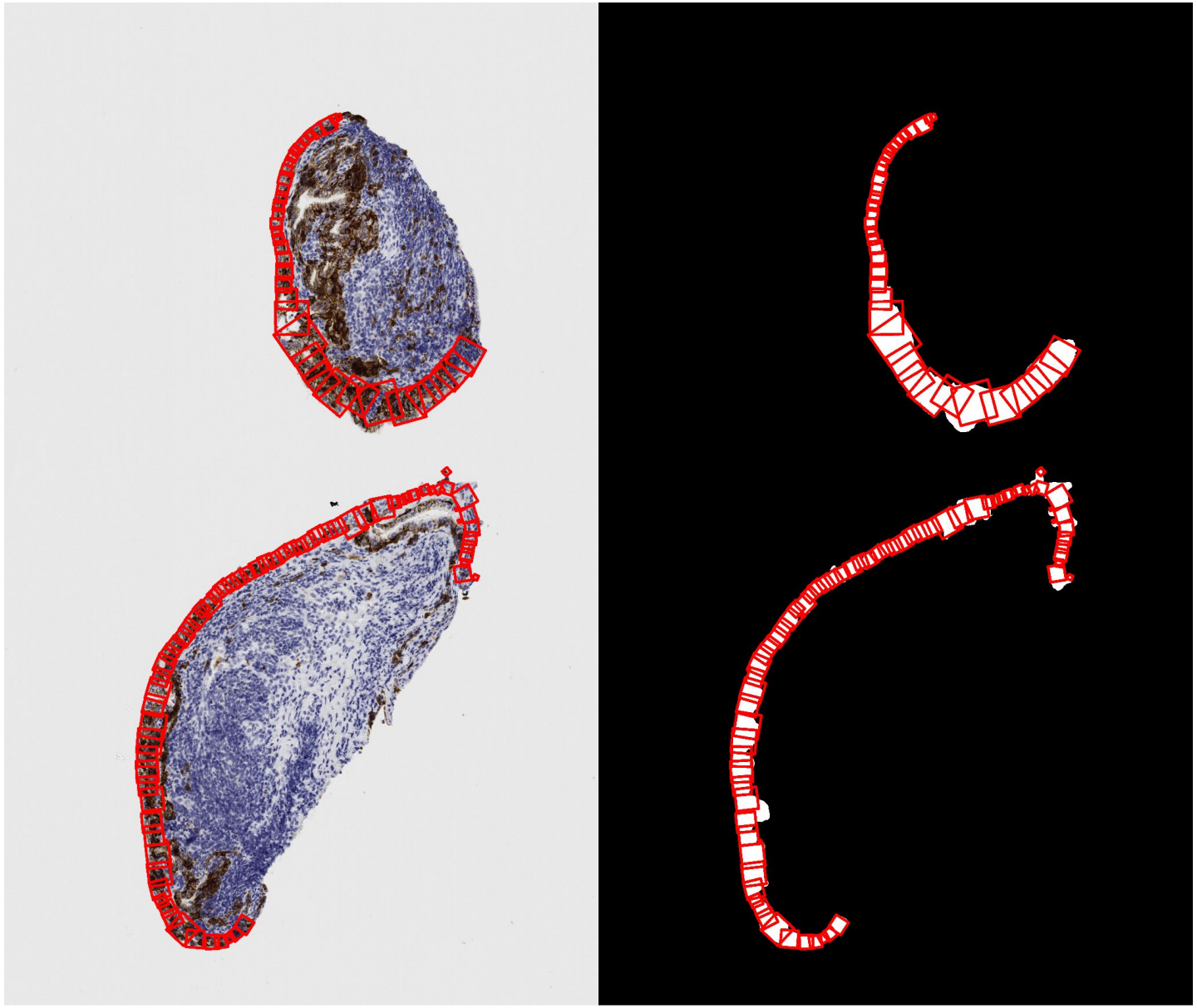
Case 8, Unmatched 1, Melan Stain



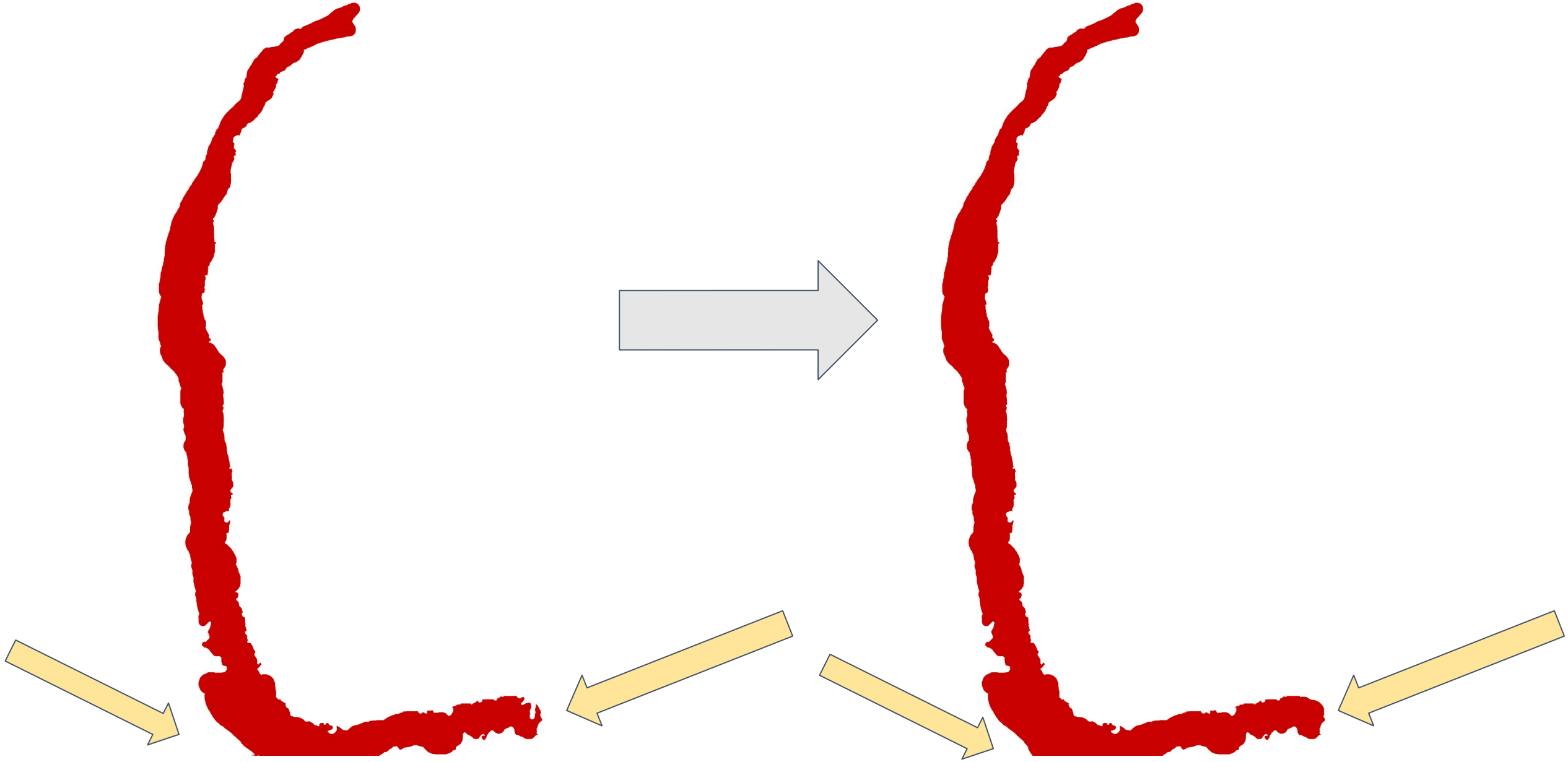
Case 8, Unmatched 2, Melan Stain



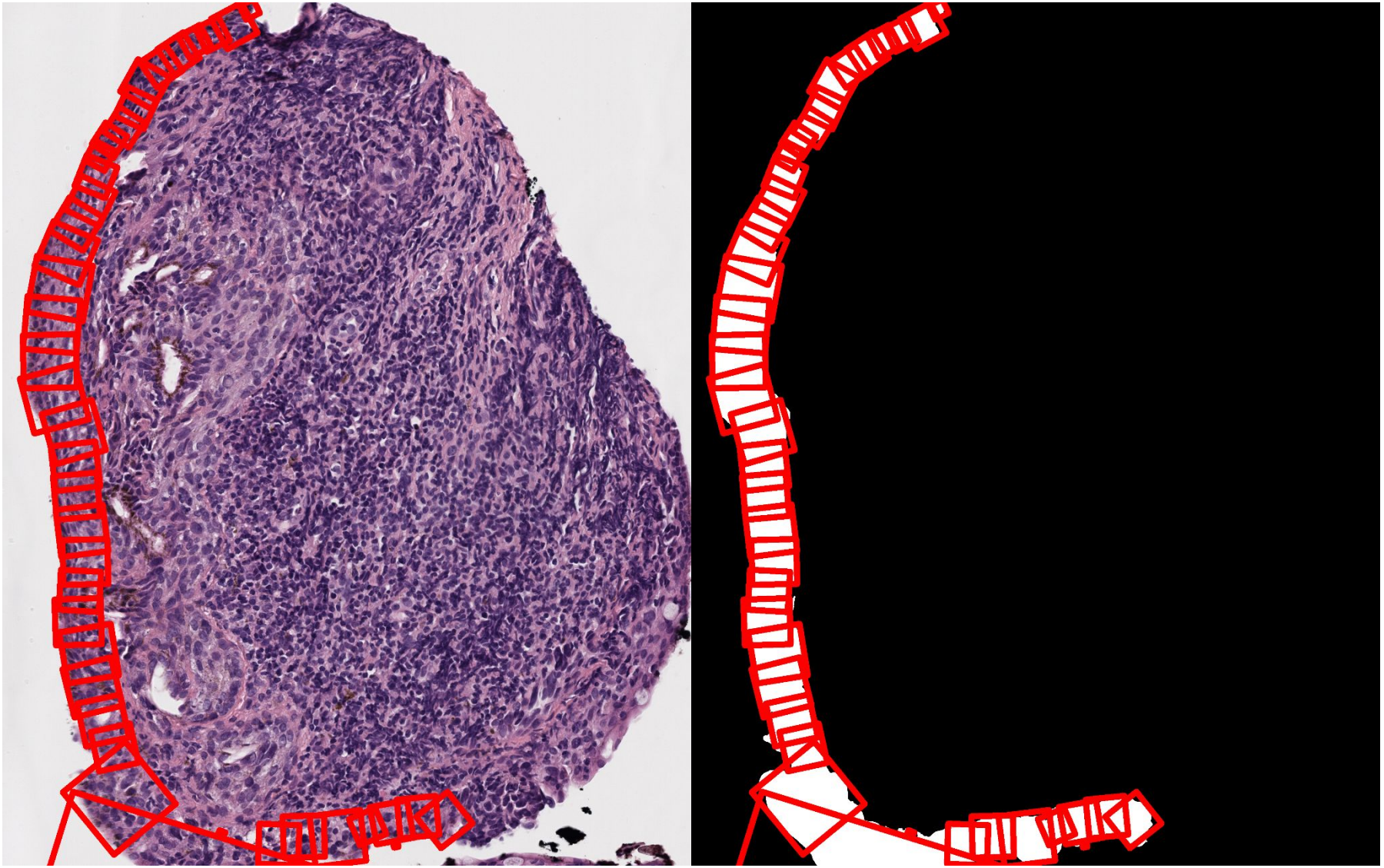
Case 8, Unmatched 2, Melan Stain



Case 8, Unmatched 3, h&e Stain



Case 8, Unmatched 3, h&e Stain



Final Notes

- One code change was required to run the patch export pipeline:

```
patch_length_filename = 'testing_slices/patch_csvs/patch_length_' + m[0].split('\\', 1)[1][:-4] + ".csv"

patch_length_filename = 'testing_slices/patch_csvs/patch_length_' + m[0].split('/', 1)[1][:-4] + ".csv"
```

- On my computer, images had to be run one at a time (with the original .tif file and the .png annotation in the testing_slices folder) in order to create patch images, but all files had to be run together to create path_outputs, otherwise the file was overwritten
  - Overwriting due to a line of code in the patch export pipeline

```
# delete any patches that are already in the directory
empty_folder()
```